

STAT 1361 Final Project Code

Elizabeth Kwee

2026-04-21

```
library(tidyverse)
library(regclass) #VIF
library(leaps) #subset selection
library(caret) #cross validation

library(glmnet) #ridge and lasso regression
library(tree) #decision tree
library(randomForest) #random forest

#Read in data
train = read.csv("df_train.csv", header = T)
test = read.csv("df_test.csv", header = T)

#View data
head(train, 5)
```

##	neighbourhood_cleansed	room_type	bathrooms	bedrooms	accommodates
## 1	Hennepin	Entire home/apt	1.0	1	5
## 2	Ramsey	Private room	1.0	1	2
## 3	Ramsey	Private room	1.0	1	2
## 4	Ramsey	Private room	1.5	1	1
## 5	Hennepin	Entire home/apt	1.0	2	4

##	minimum_nights	availability_365	number_of_reviews	review_scores_rating
## 1	28	24	298	4.80
## 2	2	350	35	5.00
## 3	2	348	16	5.00
## 4	1	325	37	4.62
## 5	14	114	6	5.00

##	review_scores_cleanliness	review_scores_location	host_is_superhost
## 1	4.72	4.80	1
## 2	5.00	5.00	1
## 3	5.00	5.00	1
## 4	3.97	4.86	1
## 5	5.00	5.00	1

##	instant_bookable	host_identity_verified	calculated_host_listings_count
## 1	0	1	2
## 2	1	1	4
## 3	1	1	4
## 4	0	1	2
## 5	0	1	3

##	dist_to_downtown	price
## 1	2.469975	91
## 2	1.008061	186

```
## 3      1.076217  163
## 4      2.814963   85
## 5      3.200156  195
```

```
##Exploratory Data Analysis
```

```
#Convert categorical variables to numeric
```

```
train.numeric = train
```

```
train.numeric$neighbourhood_cleansed = as.numeric(as.factor(train$neighbourhood_cleansed))
```

```
train.numeric$room_type = as.numeric(as.factor(train$room_type))
```

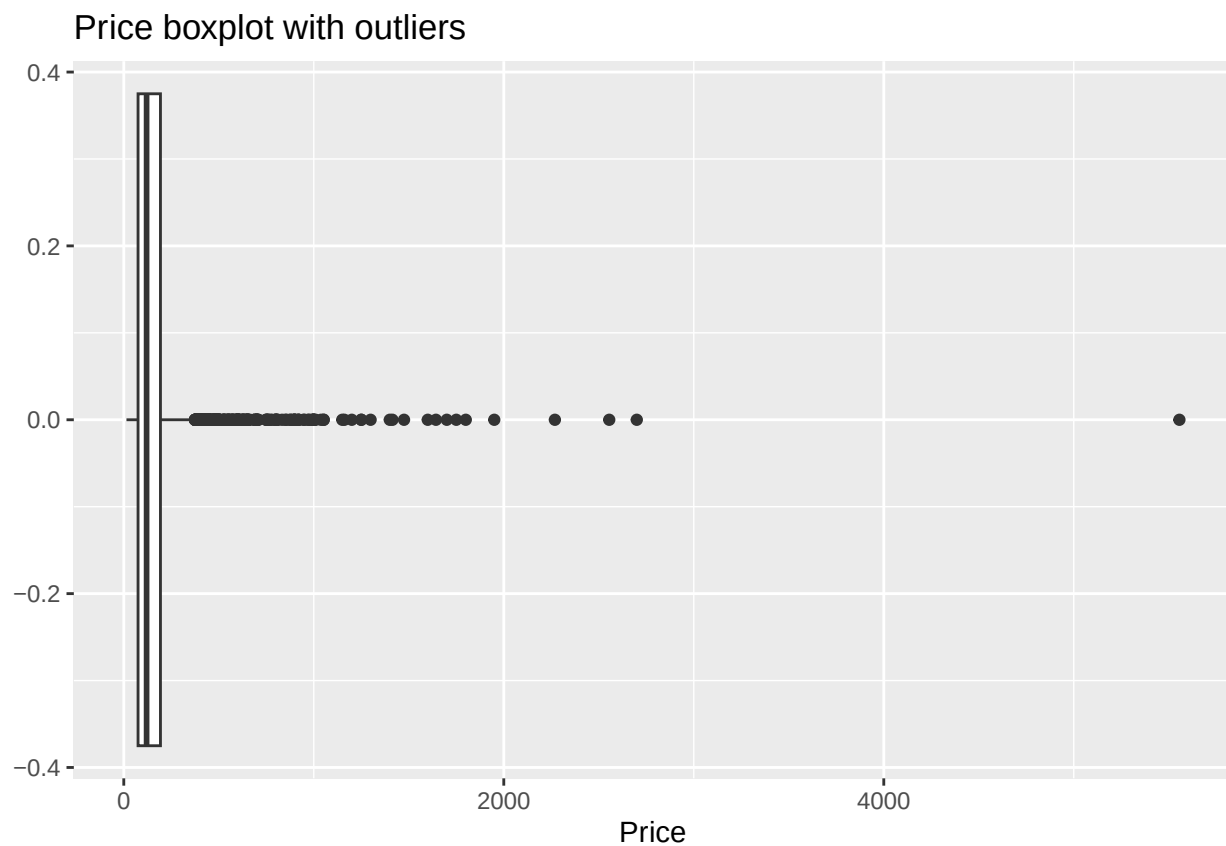
```
#Univariate Analysis
```

```
#Price (target variable)
```

```
ggplot(train, aes(x = price)) +
```

```
  geom_boxplot() +
```

```
  labs(title = "Price boxplot with outliers", x = "Price")
```

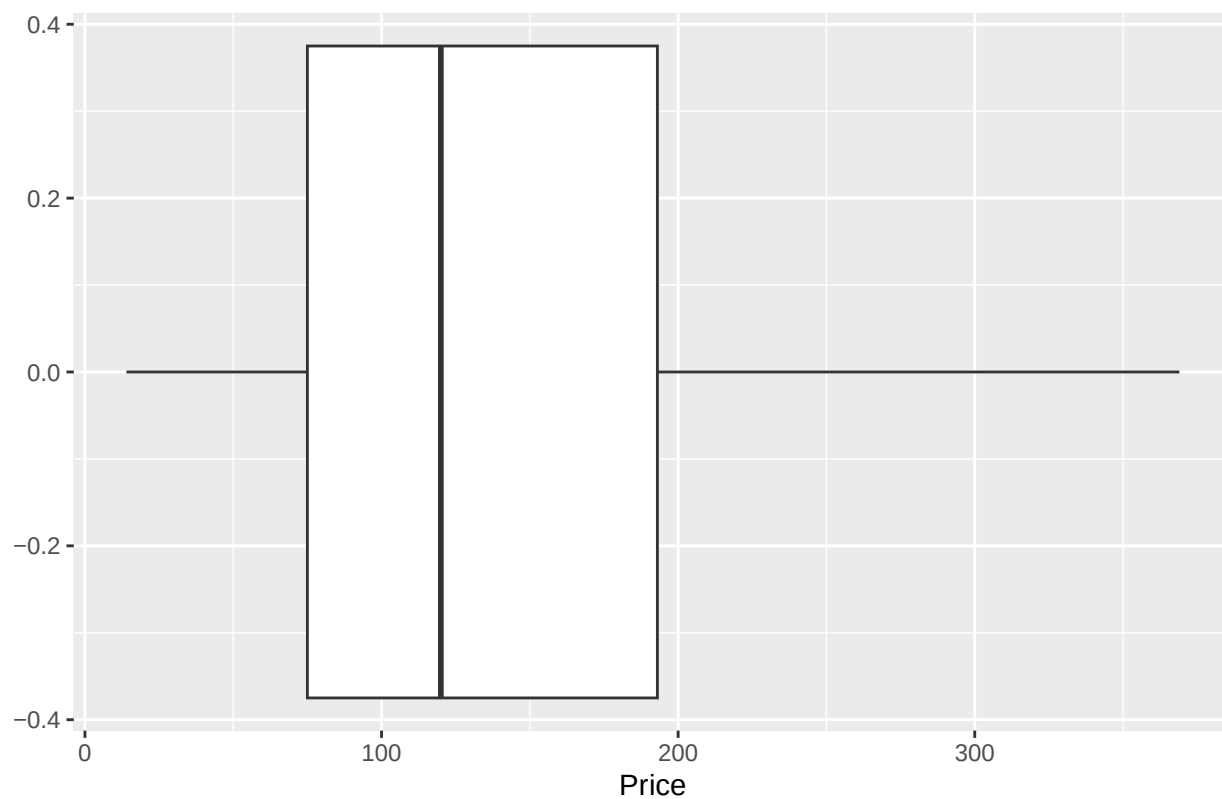


```
ggplot(train, aes(x = price)) +
```

```
  geom_boxplot(outliers = F) +
```

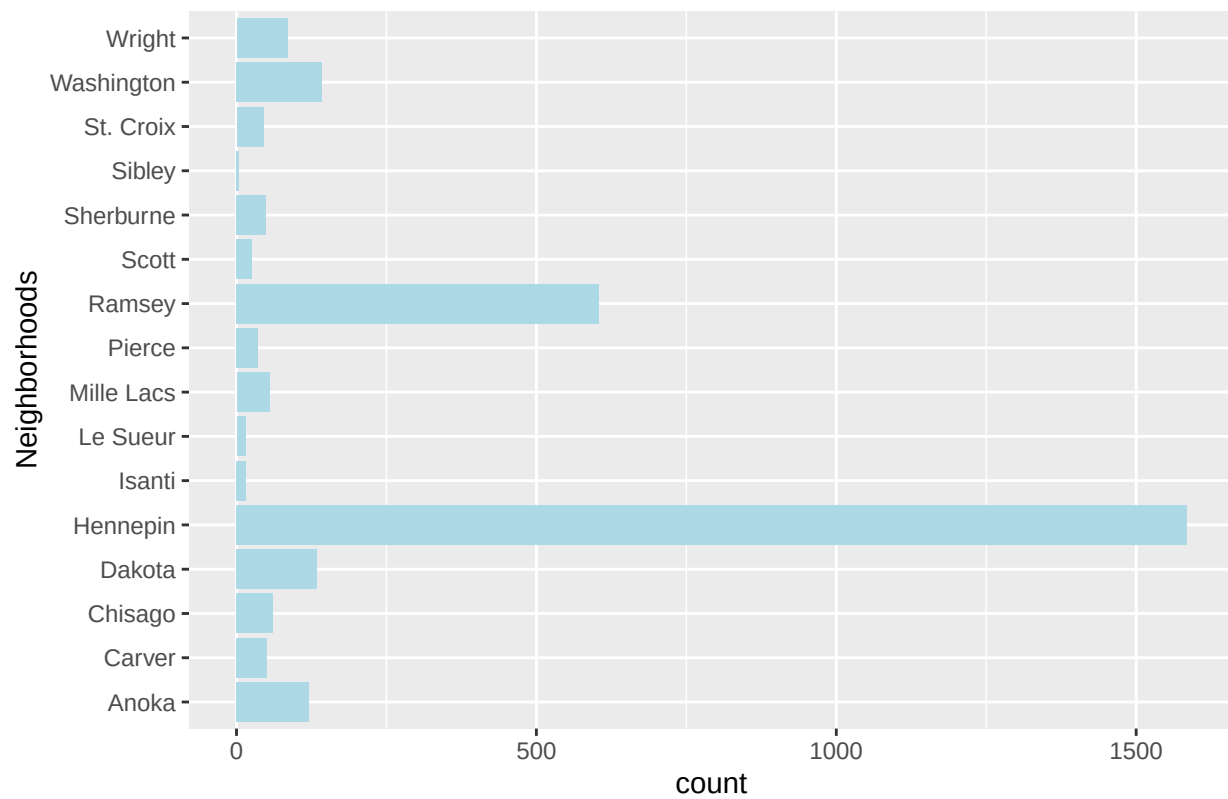
```
  labs(title = "Price boxplot without outliers", x = "Price")
```

Price boxplot without outliers



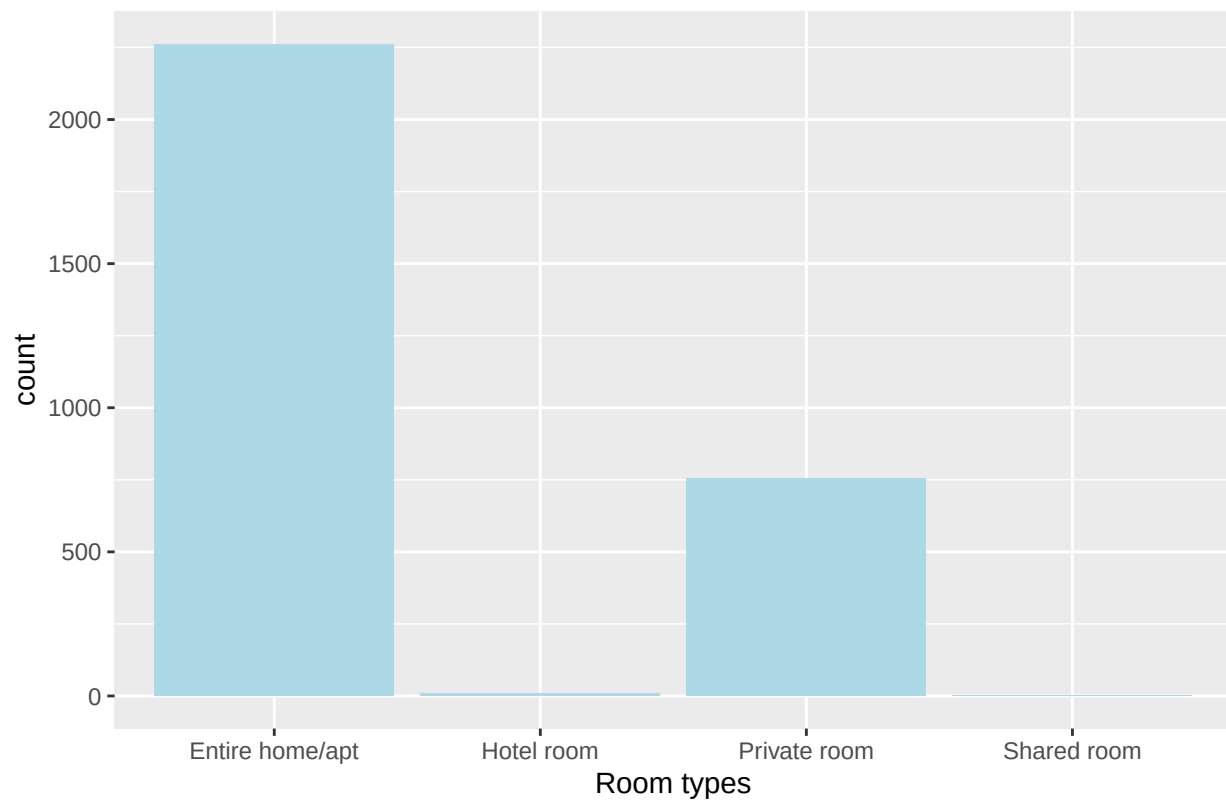
```
#Neighbourhood_cleansed  
ggplot(train, aes(x=neighbourhood_cleansed)) +  
  geom_bar(fill="lightblue") +  
  coord_flip() +  
  labs(title = "Bar chart of neighborhoods", x = "Neighborhoods")
```

Bar chart of neighborhoods

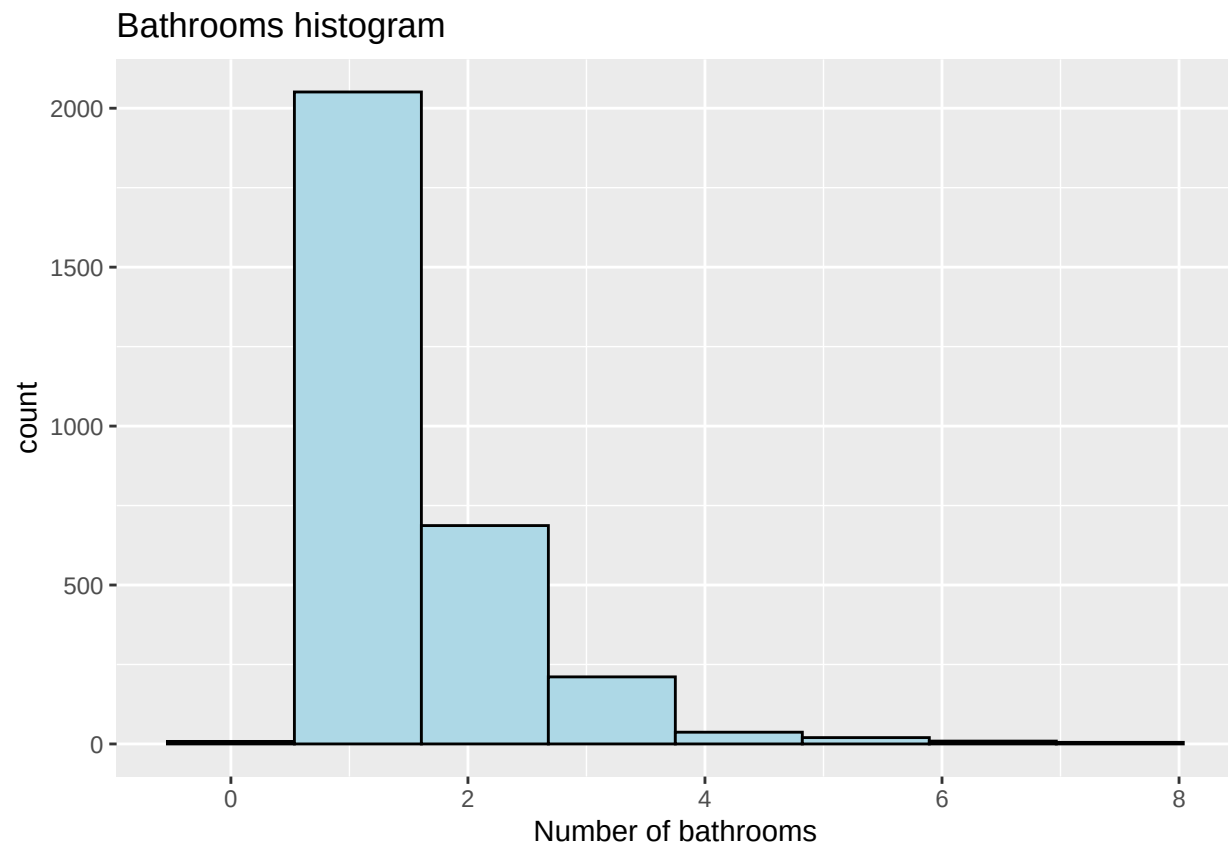


```
#Room type
ggplot(train, aes(x=room_type)) +
  geom_bar(fill="lightblue") +
  labs(title = "Bar chart of room types", x = "Room types")
```

Bar chart of room types

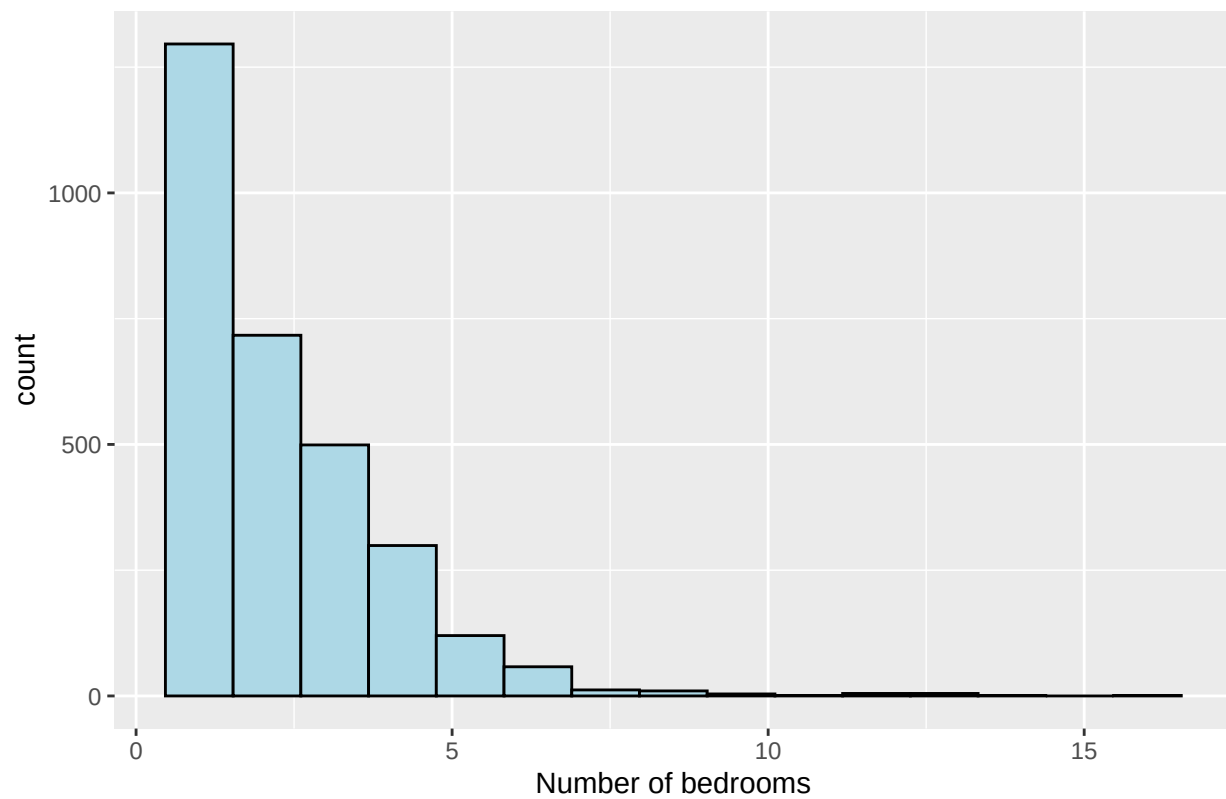


```
#Bathrooms  
ggplot(train, aes(x = bathrooms)) +  
  geom_histogram(bins = 8, fill="lightblue", col="black") +  
  labs(title = "Bathrooms histogram", x = "Number of bathrooms")
```



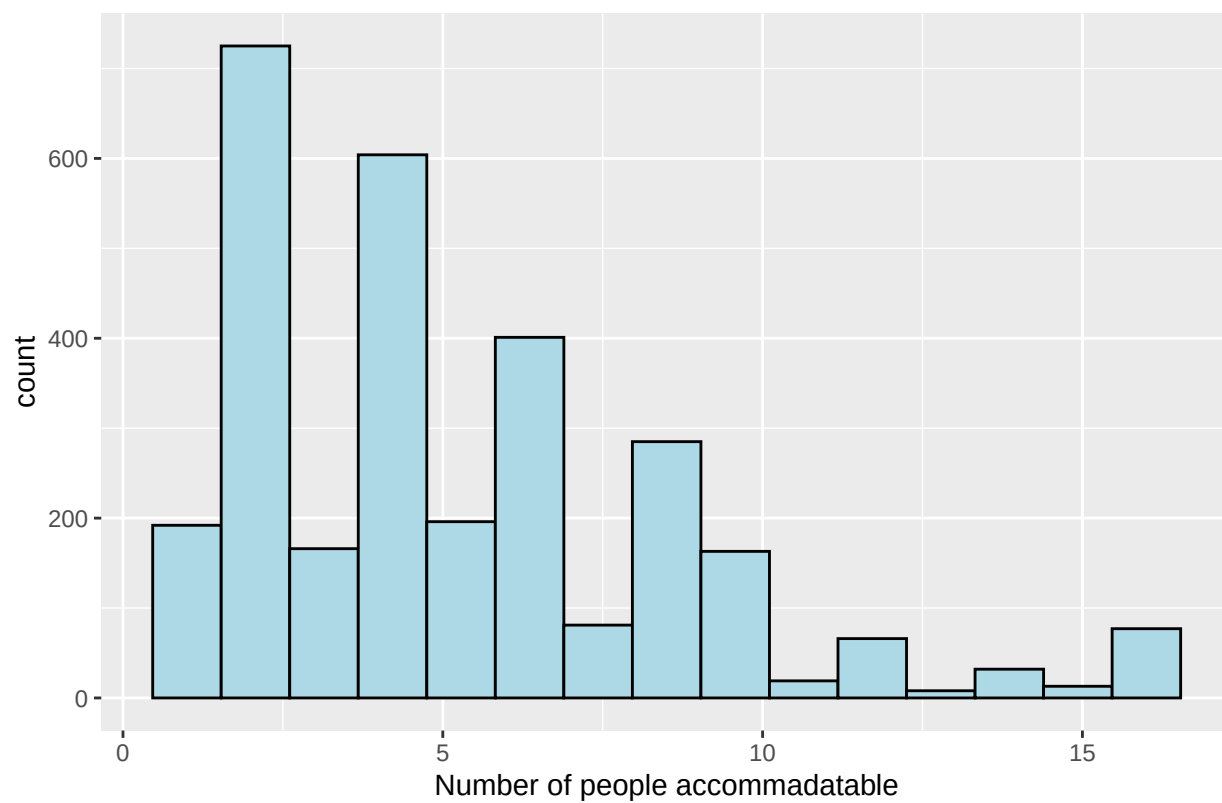
```
#Bedrooms  
ggplot(train, aes(x = bedrooms)) +  
  geom_histogram(bins = 15, fill="lightblue", col="black") +  
  labs(title = "Bedrooms histogram", x = "Number of bedrooms")
```

Bedrooms histogram

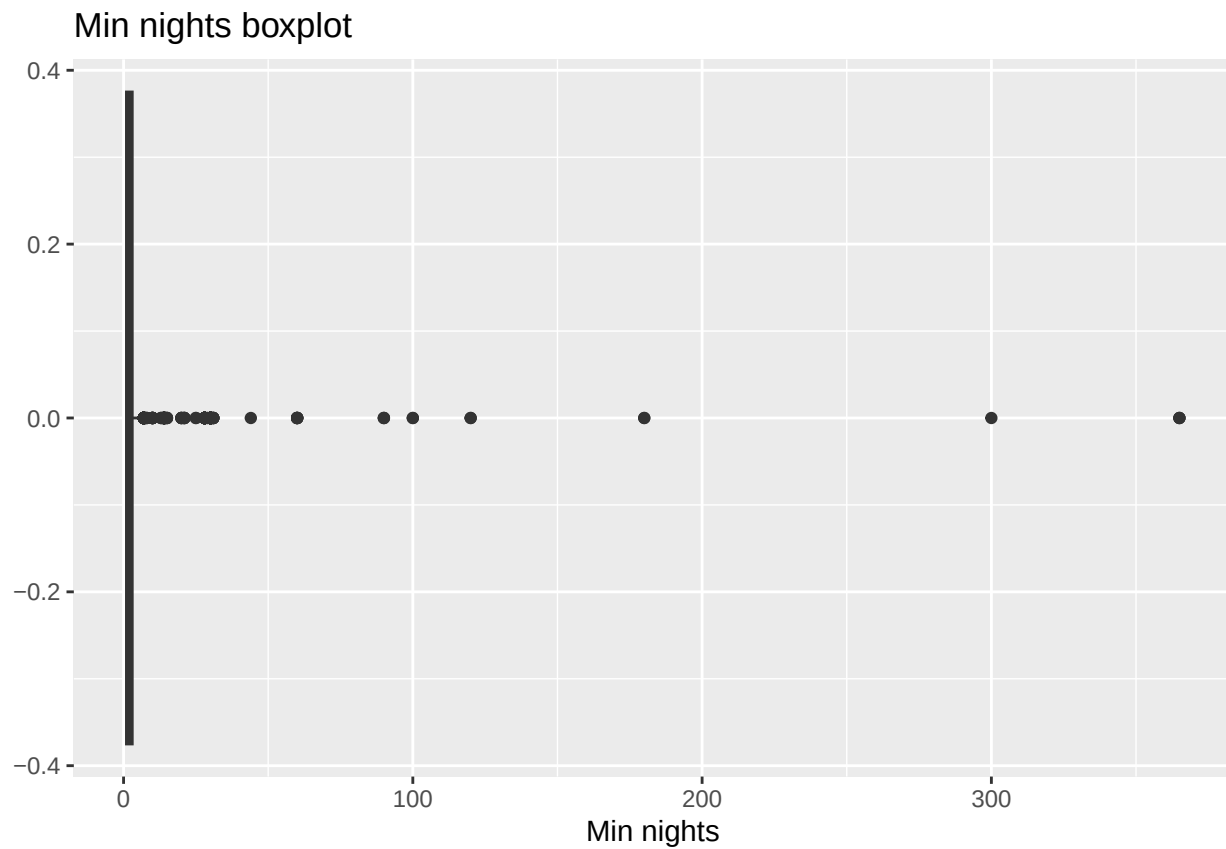


```
#Accommodates  
ggplot(train, aes(x = accommodates)) +  
  geom_histogram(bins = 15, fill="lightblue", col="black") +  
  labs(title = "Accommodates histogram", x = "Number of people accommadatable")
```

Accommodates histogram

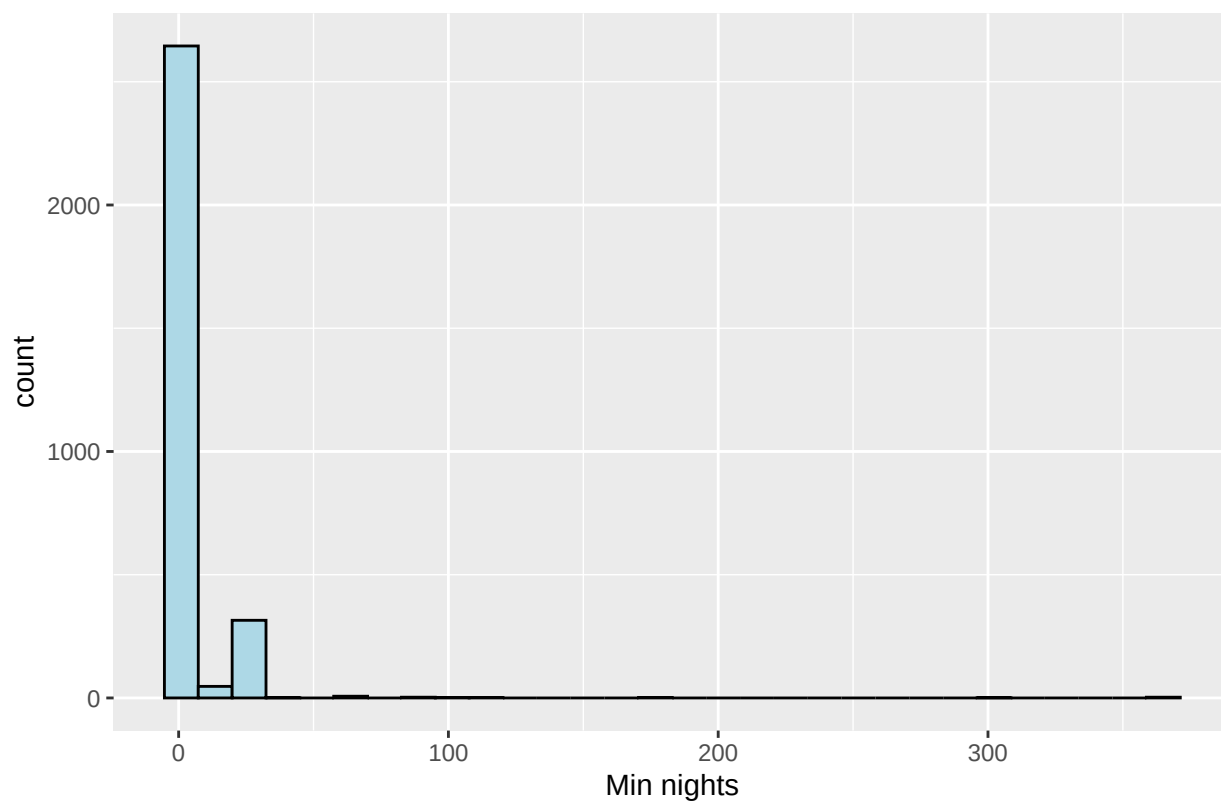


```
#Minimum_nights  
ggplot(train, aes(x = minimum_nights)) +  
  geom_boxplot() +  
  labs(title = "Min nights boxplot", x = "Min nights")
```

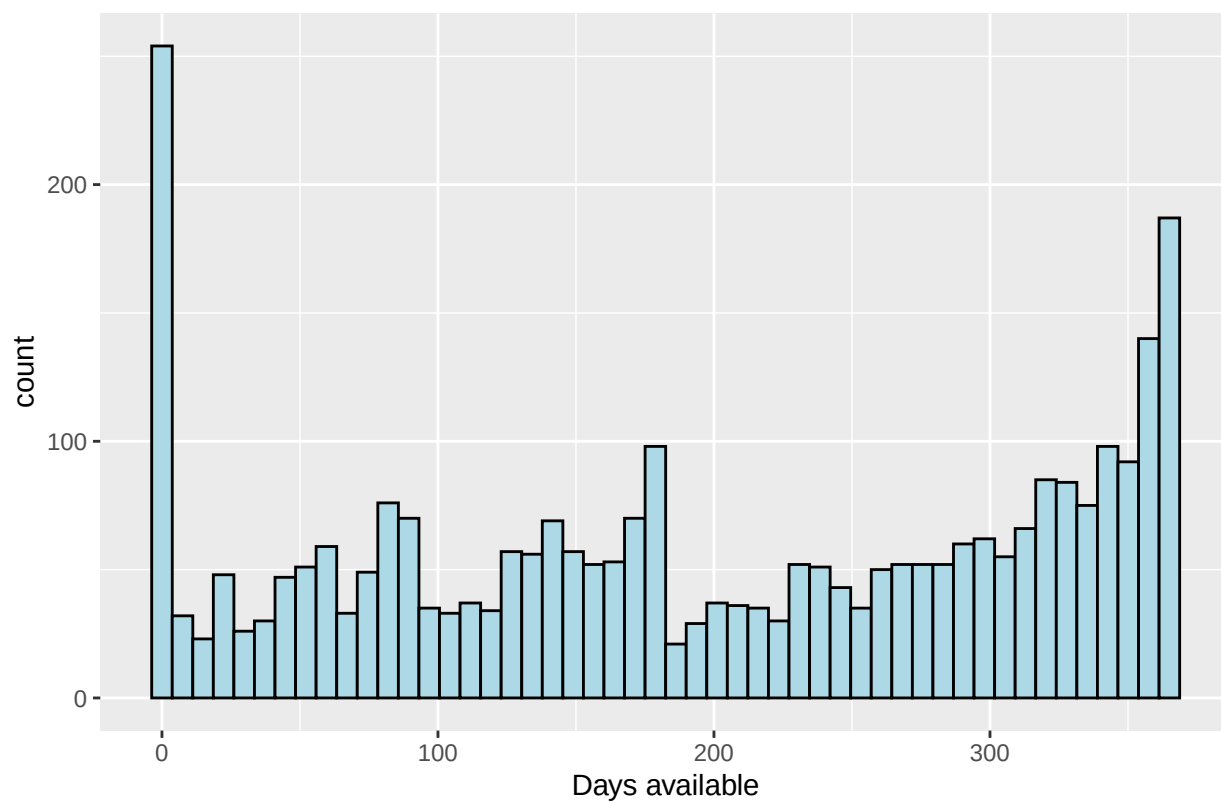
```
ggplot(train, aes(x = minimum_nights)) +  
  geom_histogram(fill="lightblue", col="black") +  
  labs(title = "Min nights histogram", x = "Min nights")
```

Min nights histogram



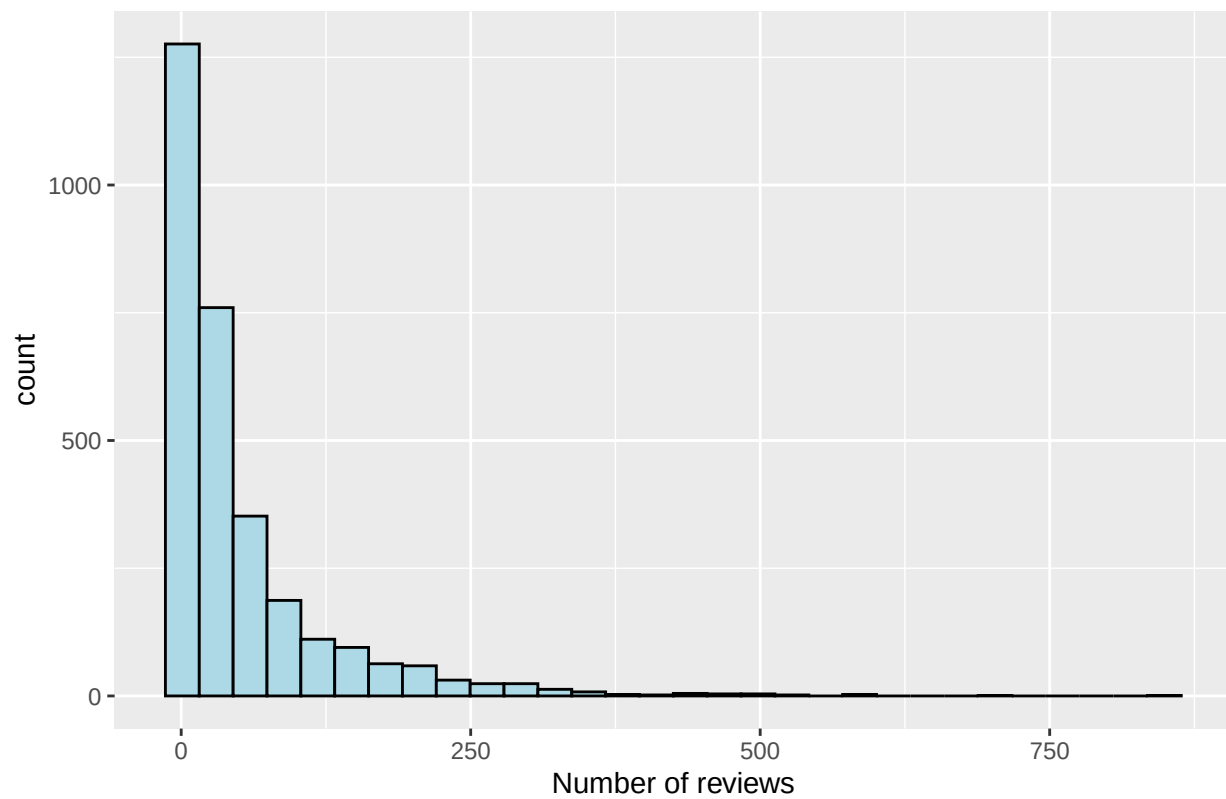
```
#Availability_365  
ggplot(train, aes(x = availability_365)) +  
  geom_histogram(bins = 50, fill="lightblue", col="black") +  
  labs(title = "Availability in a year histogram", x = "Days available")
```

Availability in a year histogram



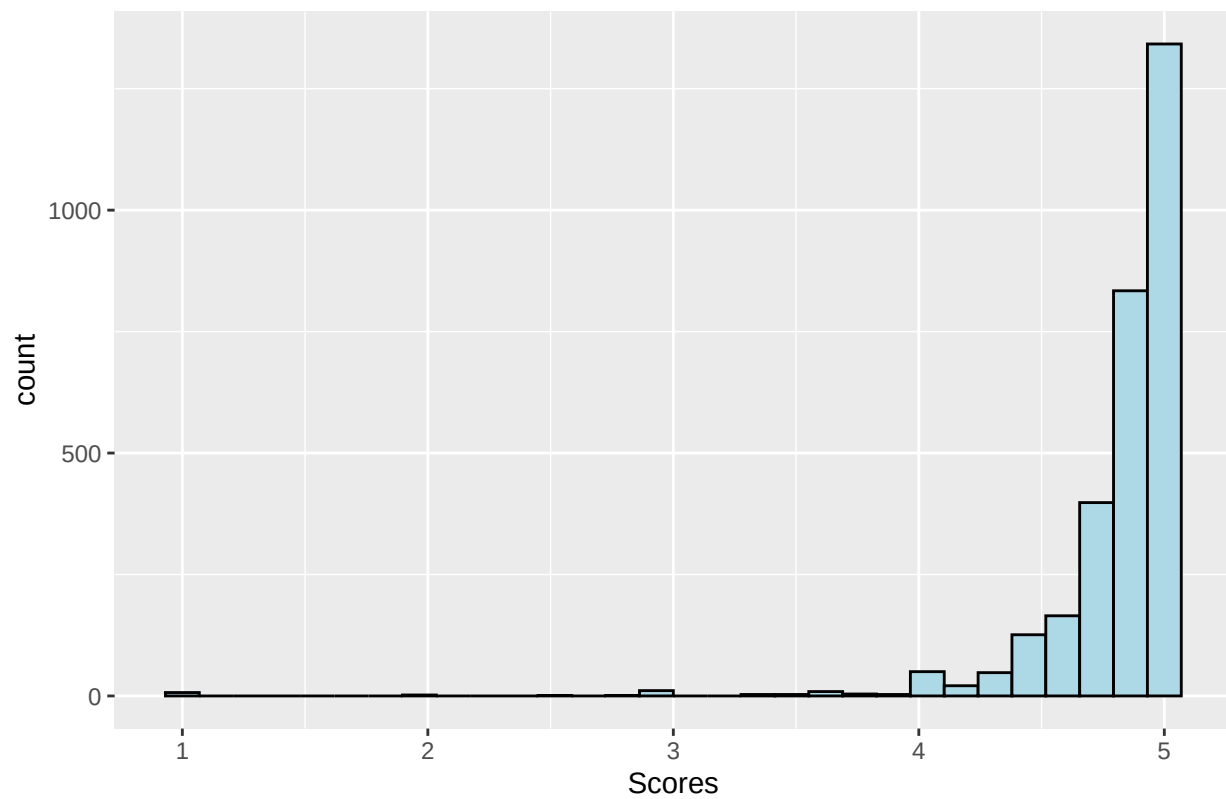
```
#Number_of_reviews  
ggplot(train, aes(x = number_of_reviews)) +  
  geom_histogram(fill = "lightblue", col = "black") +  
  labs(title = "Number of reviews histogram", x = "Number of reviews")
```

Number of reviews histogram



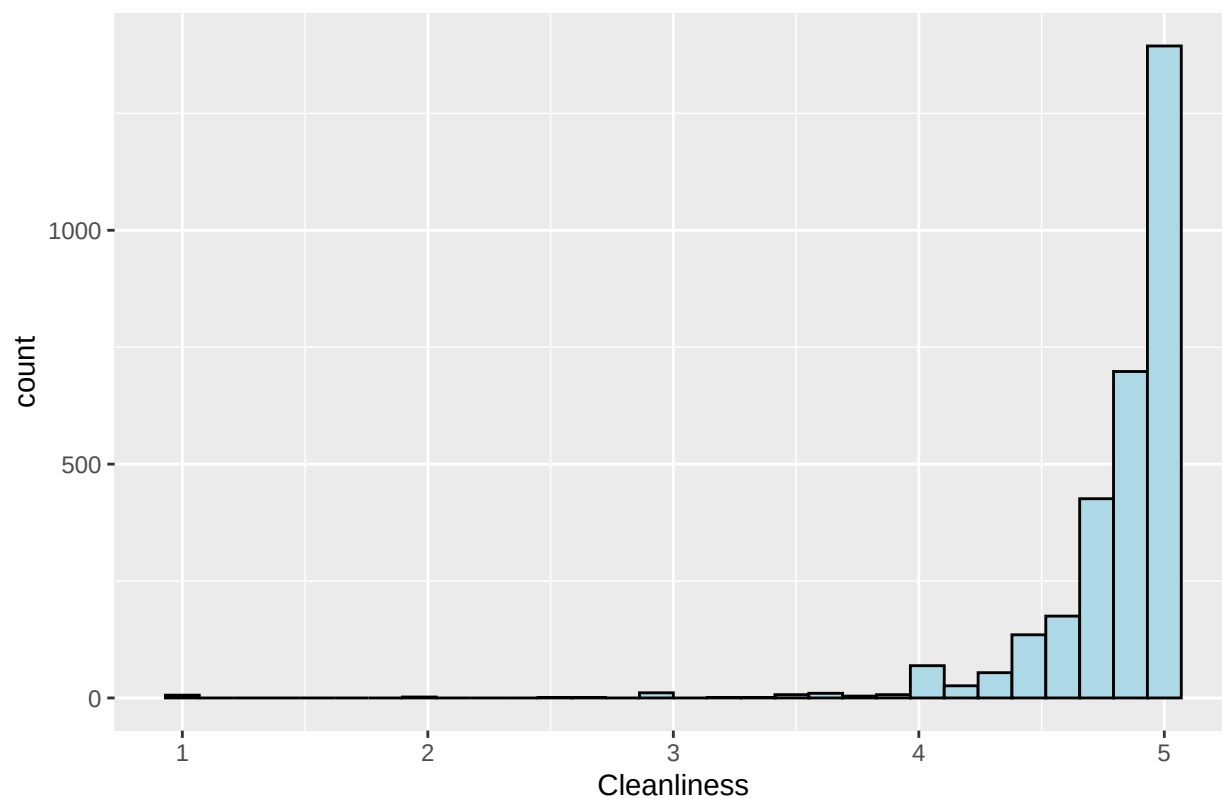
```
#Review_scores_rating  
ggplot(train, aes(x = review_scores_rating)) +  
  geom_histogram(fill = "lightblue", col = "black") +  
  labs(title = "Review scores histogram", x = "Scores")
```

Review scores histogram



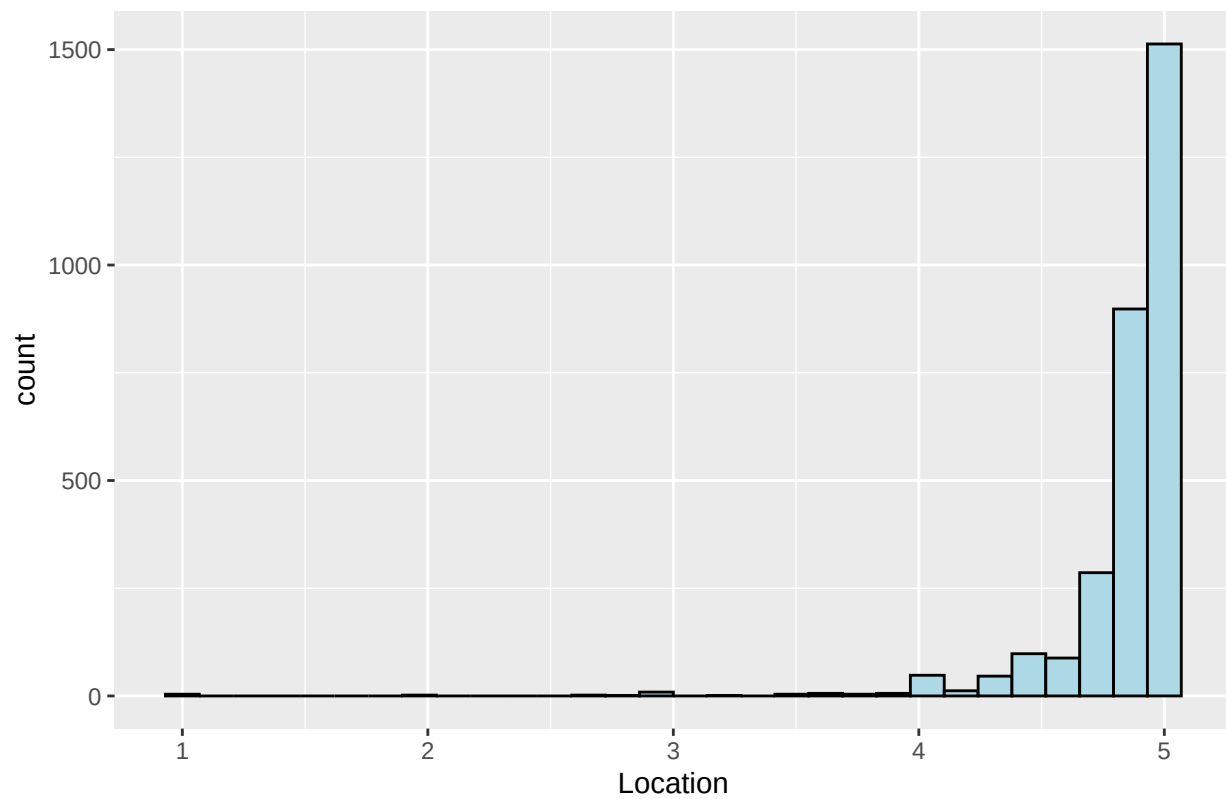
```
#Review_scores_cleanliness  
ggplot(train, aes(x = review_scores_cleanliness)) +  
  geom_histogram(fill = "lightblue", col = "black") +  
  labs(title = "Review scores cleanliness histogram", x = "Cleanliness")
```

Review scores cleanliness histogram

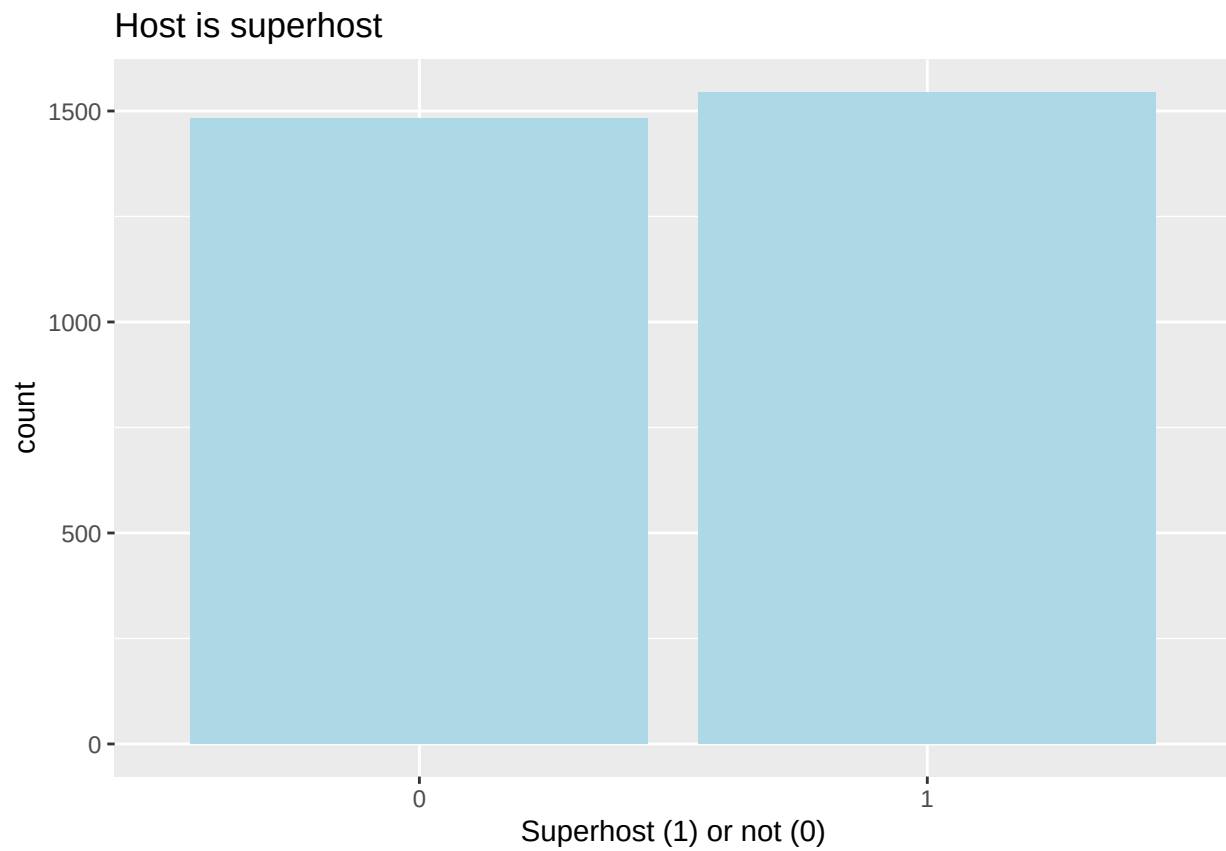


```
#Review_scores_location  
ggplot(train, aes(x = review_scores_location)) +  
  geom_histogram(fill = "lightblue", col = "black") +  
  labs(title = "Review scores location histogram", x = "Location")
```

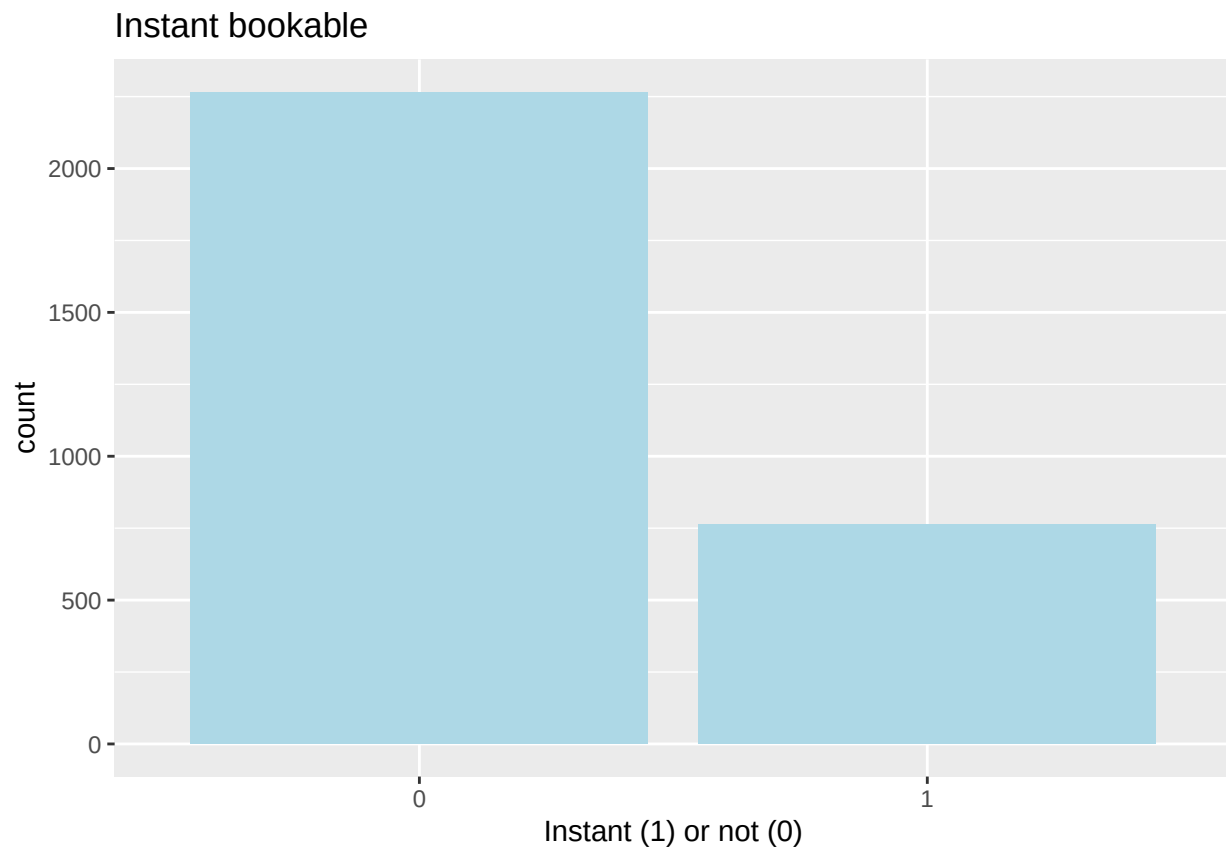
Review scores location histogram



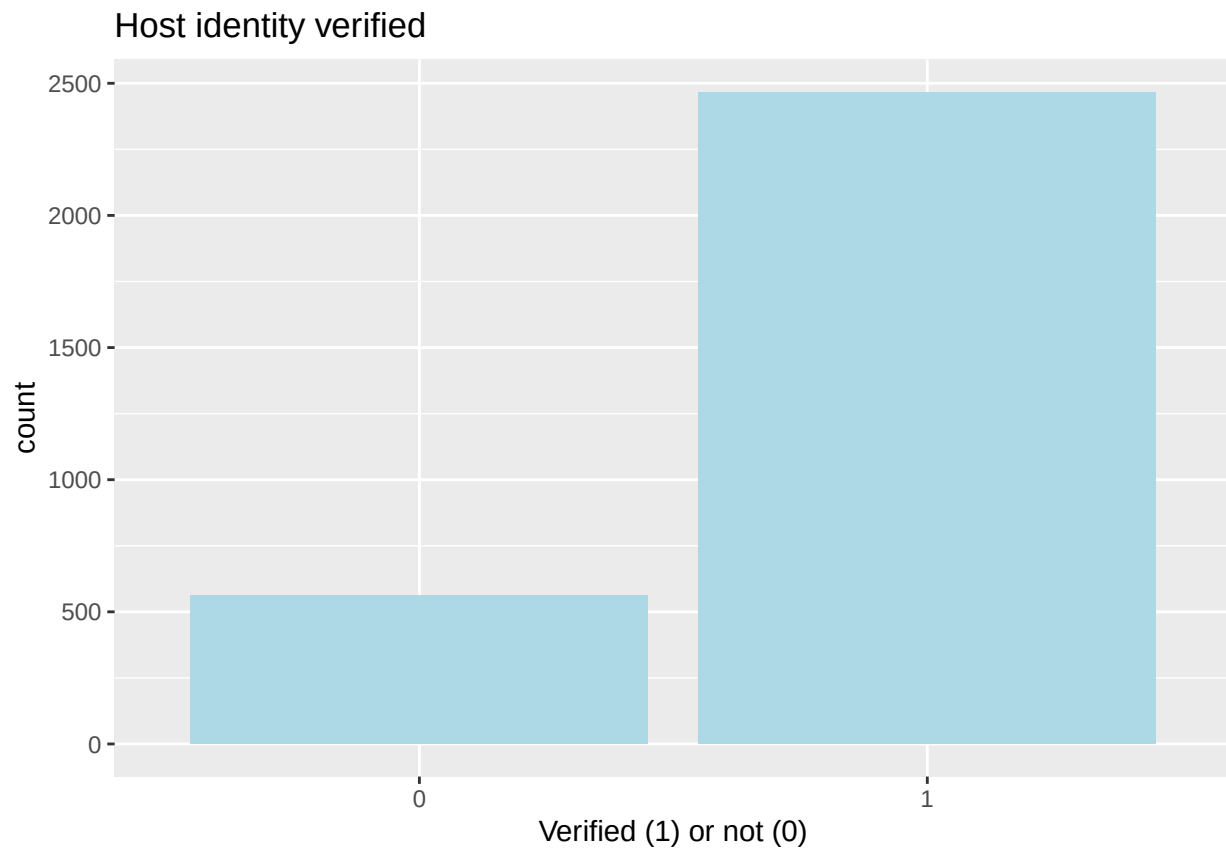
```
#Host_is_superhost  
ggplot(train, aes(x = as.factor(host_is_superhost))) +  
  geom_bar(fill = "lightblue") +  
  labs(title = "Host is superhost", x = "Superhost (1) or not (0)")
```



```
#Instant_bookable  
ggplot(train, aes(x = as.factor(instant_bookable))) +  
  geom_bar(fill = "lightblue") +  
  labs(title = "Instant bookable", x = "Instant (1) or not (0)")
```

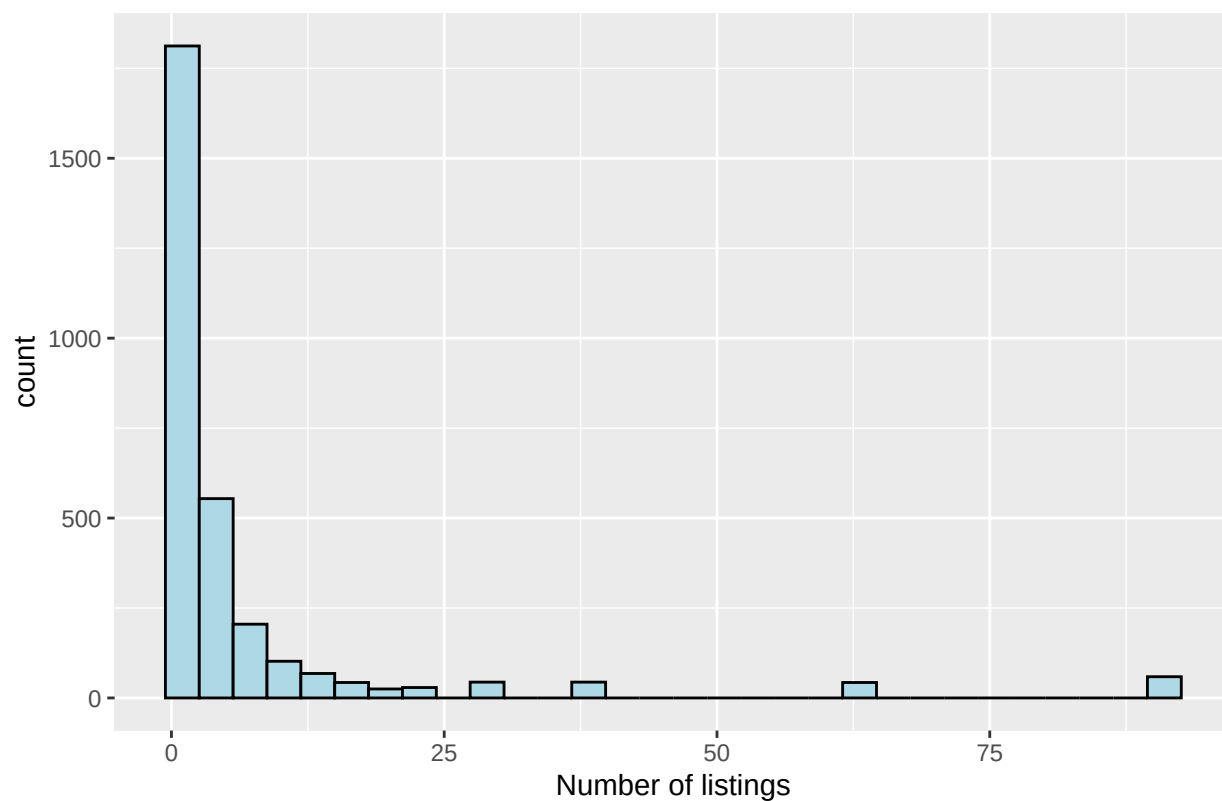



```
#Host_identity_verified  
ggplot(train, aes(x = as.factor(host_identity_verified))) +  
  geom_bar(fill = "lightblue") +  
  labs(title = "Host identity verified", x = "Verified (1) or not (0)")
```

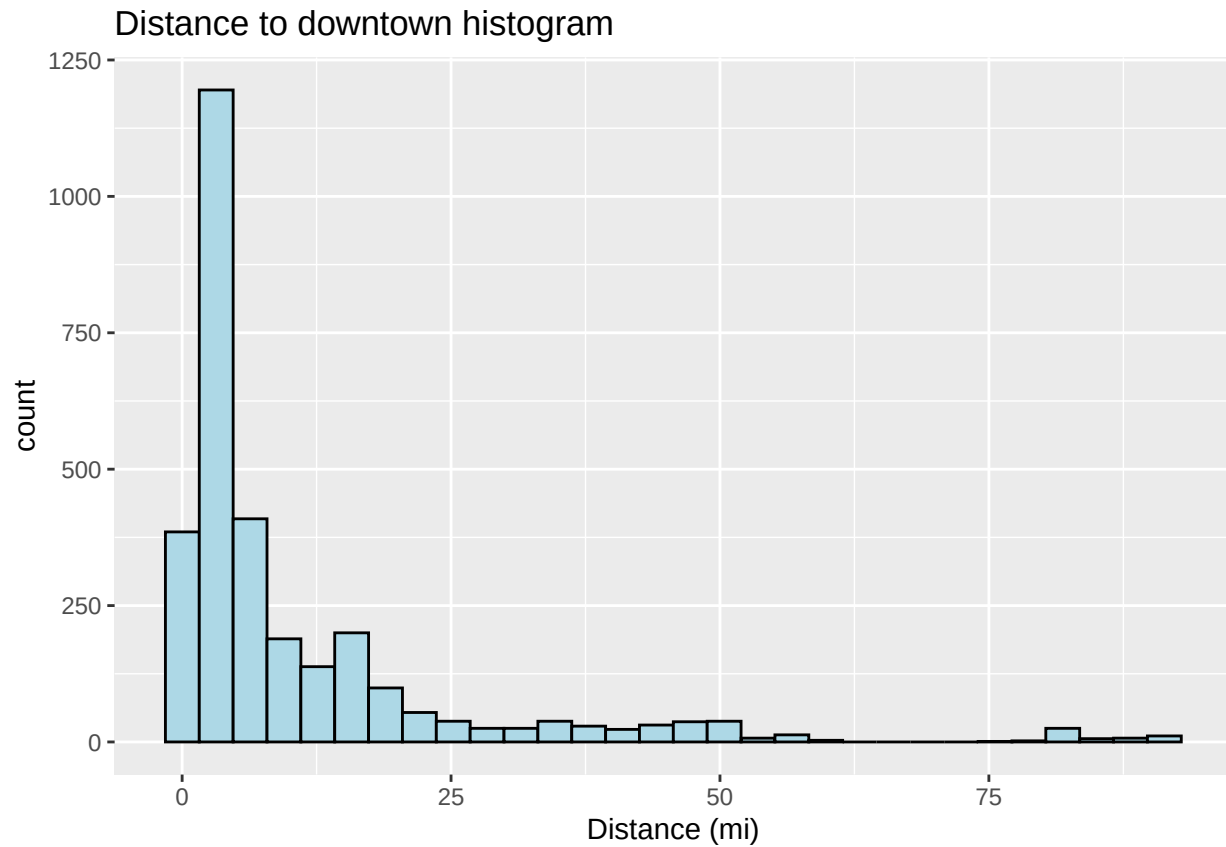


```
#Calculated_host_listings_count  
ggplot(train, aes(x = calculated_host_listings_count)) +  
  geom_histogram(fill = "lightblue", col = "black") +  
  labs(title = "Number of host listings histogram", x = "Number of listings")
```

Number of host listings histogram



```
#Dist_to_downtown  
ggplot(train, aes(x = dist_to_downtown)) +  
  geom_histogram(fill = "lightblue", col = "black") +  
  labs(title = "Distance to downtown histogram", x = "Distance (mi)")
```



```
summary(train$price)
```

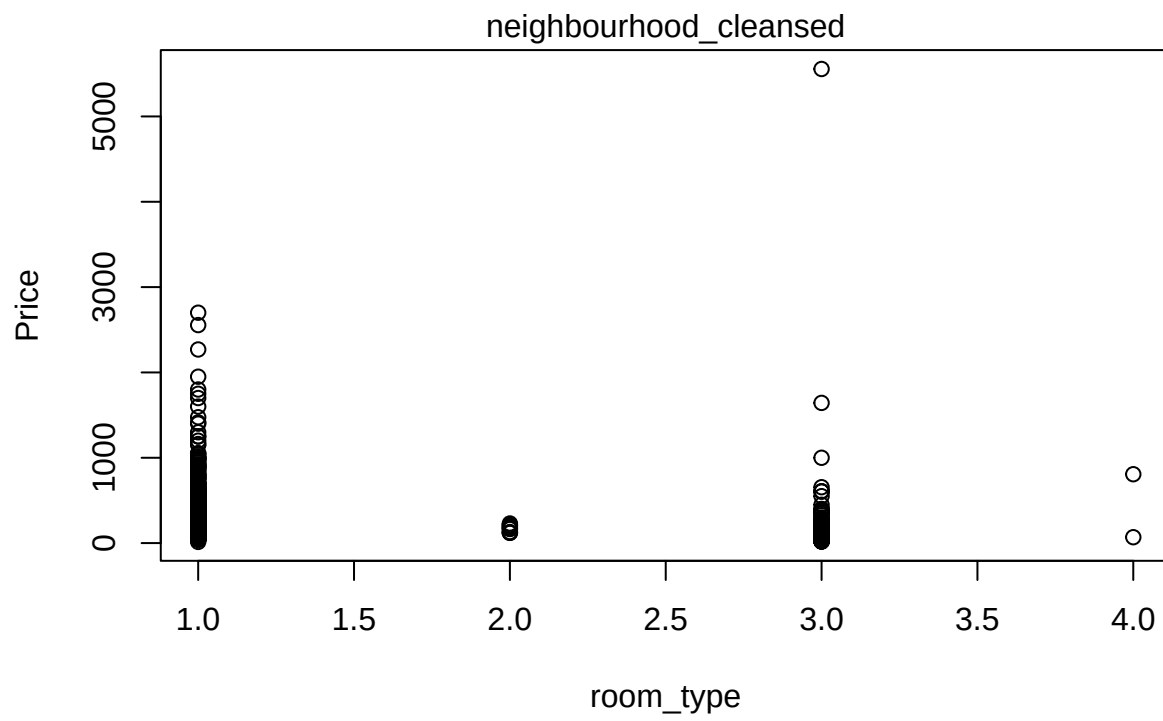
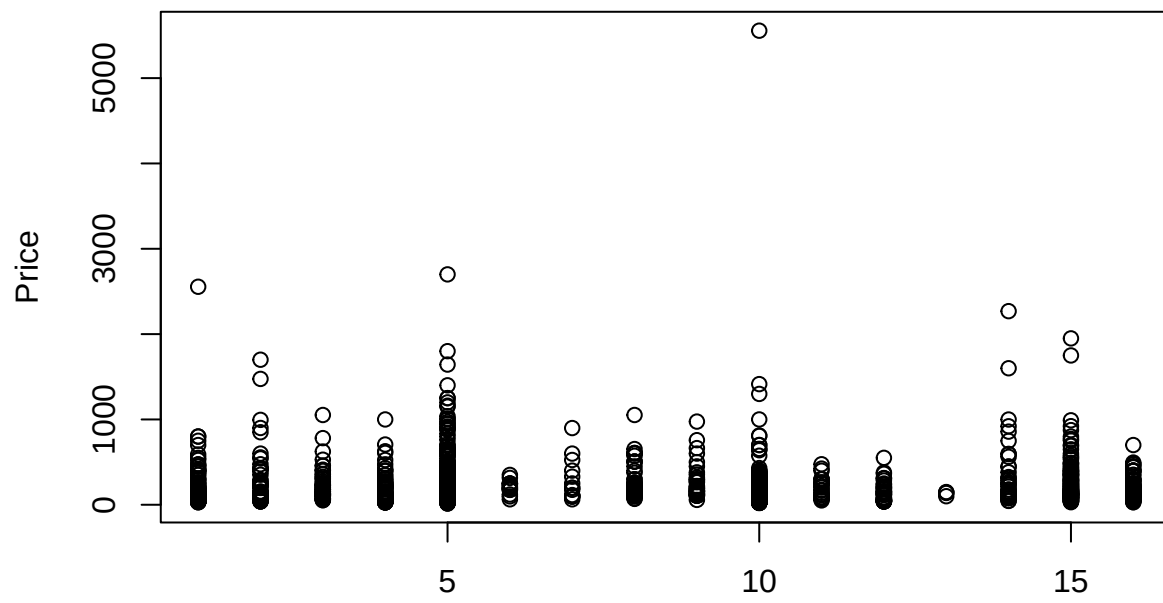
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      14.0   75.0   120.0   170.3   193.0  5556.0
```

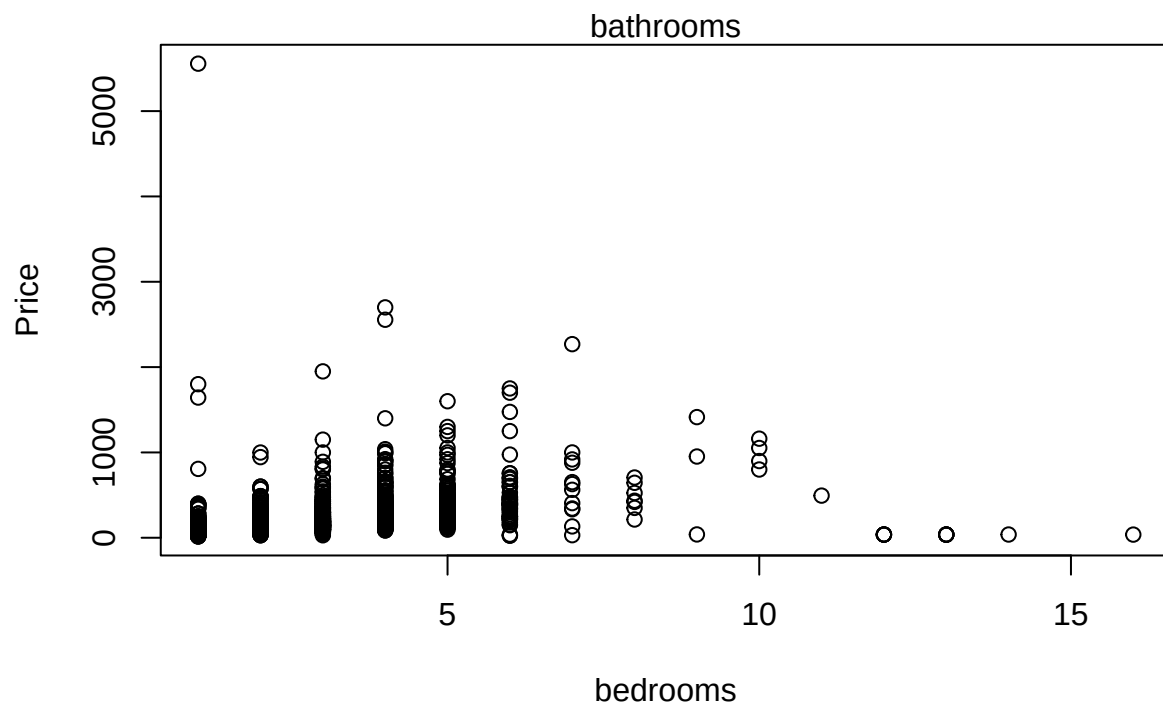
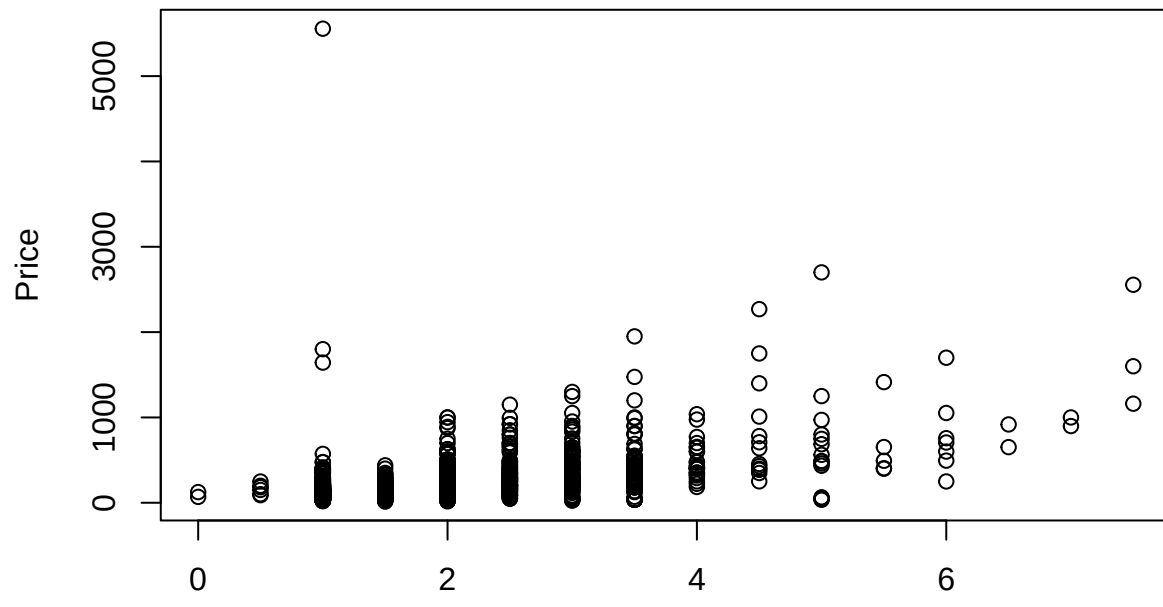
```
outlier_values = boxplot.stats(train$price)$out
length(outlier_values)
```

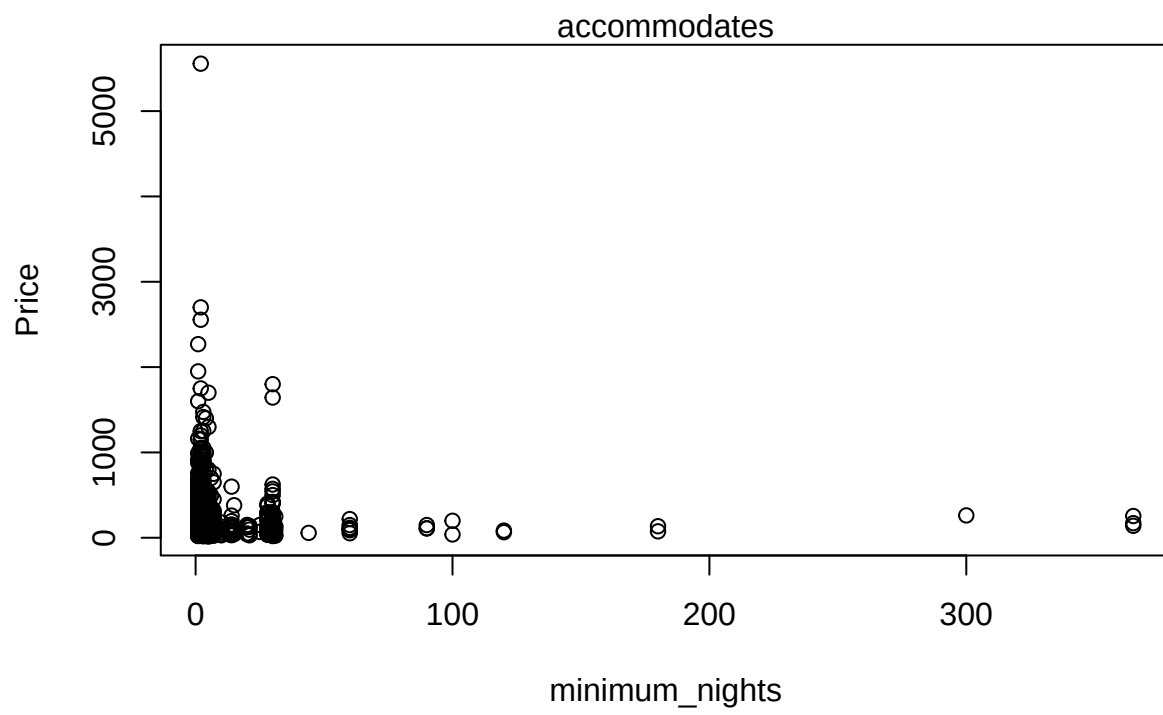
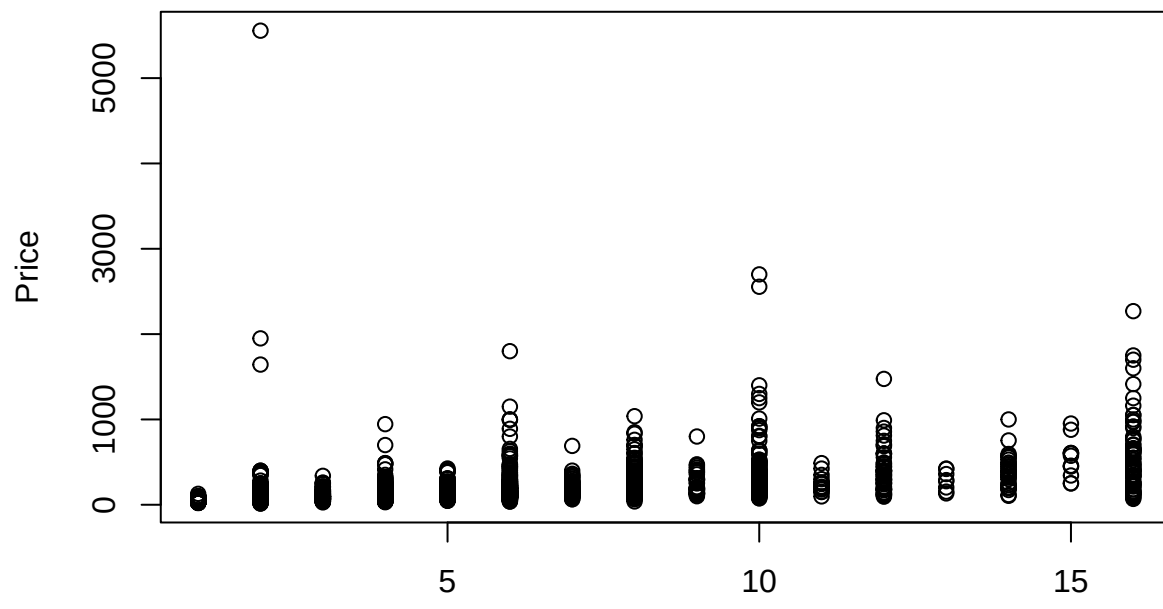
```
## [1] 250
```

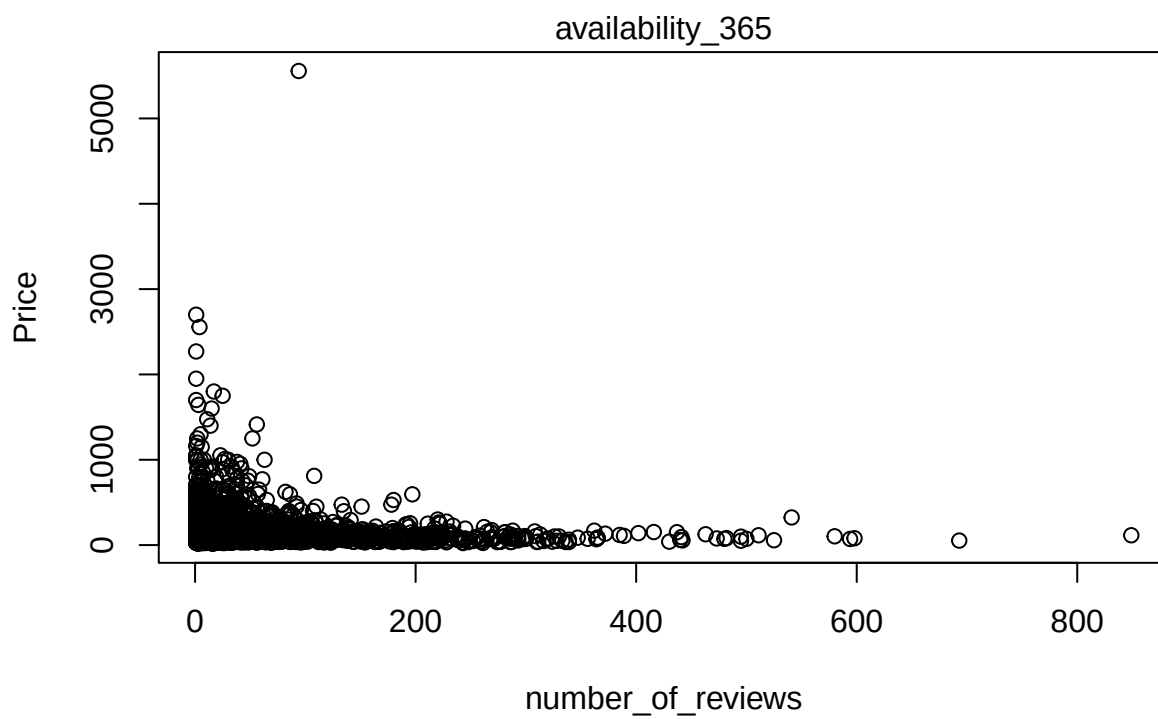
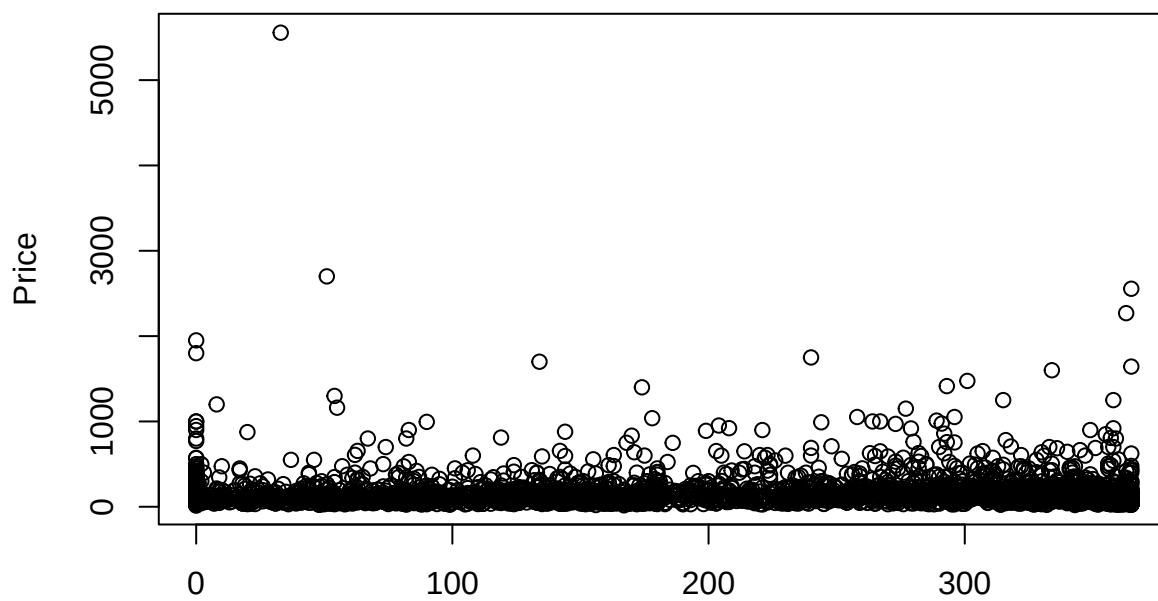
```
##Bivariate analysis #Price vs. variables
```

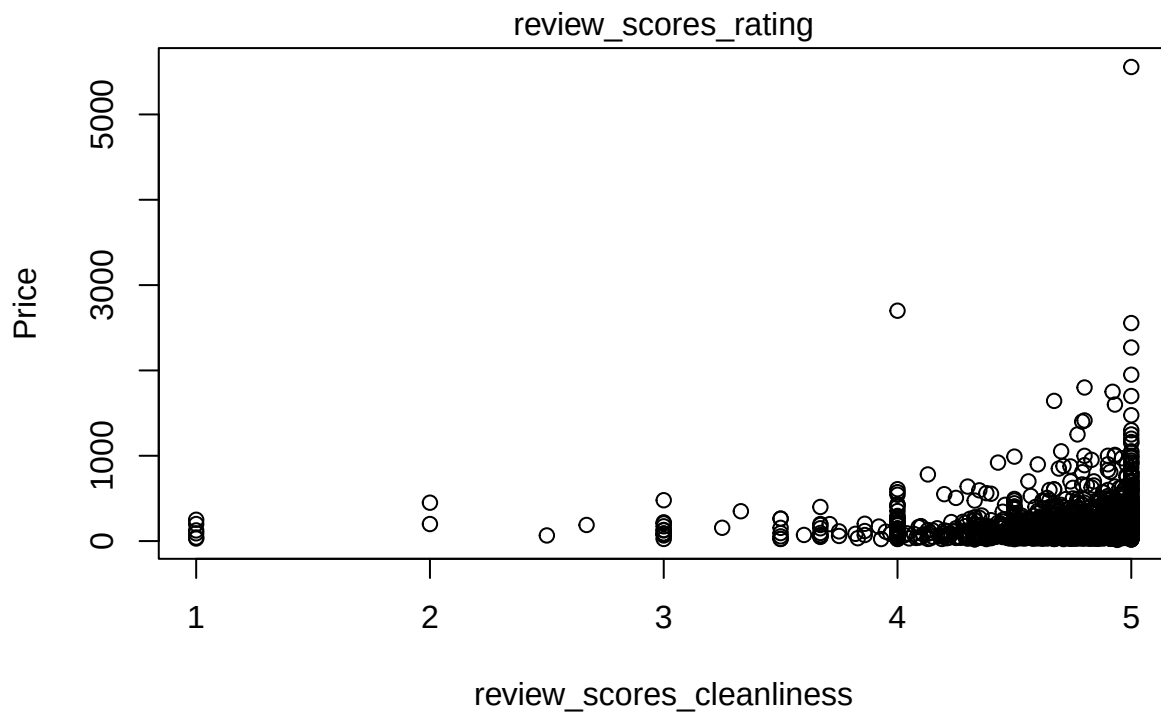
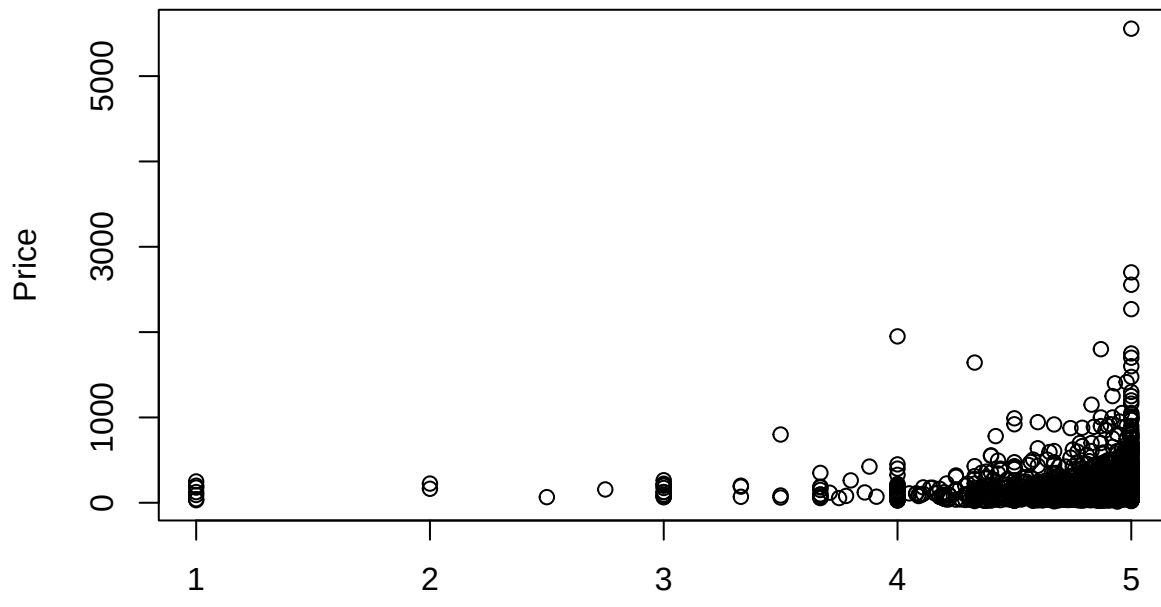
```
for (i in 1:16){
  plot(train.numeric[,i], train.numeric$price, xlab = colnames(train.numeric)[i], ylab="Price")
}
```

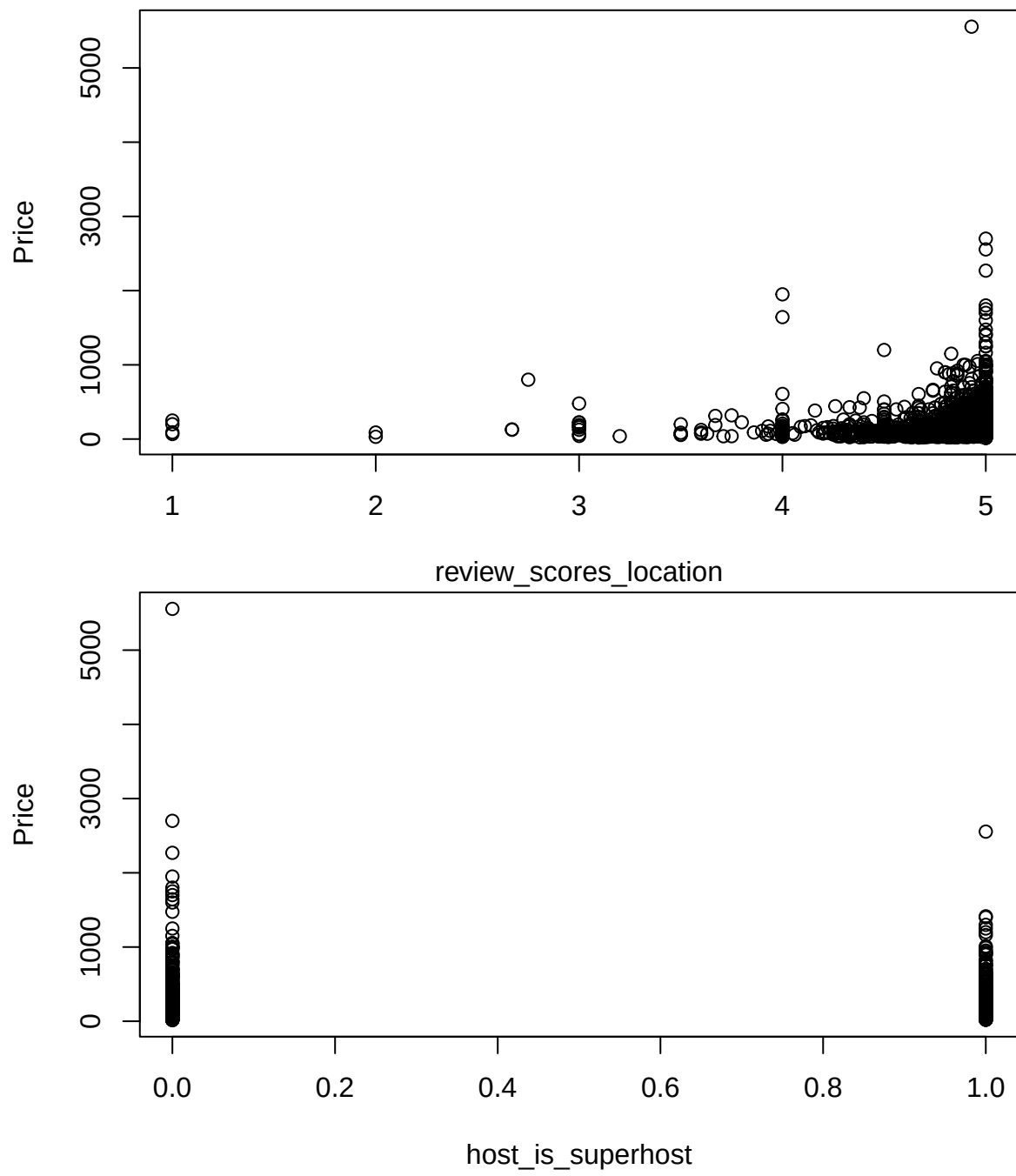


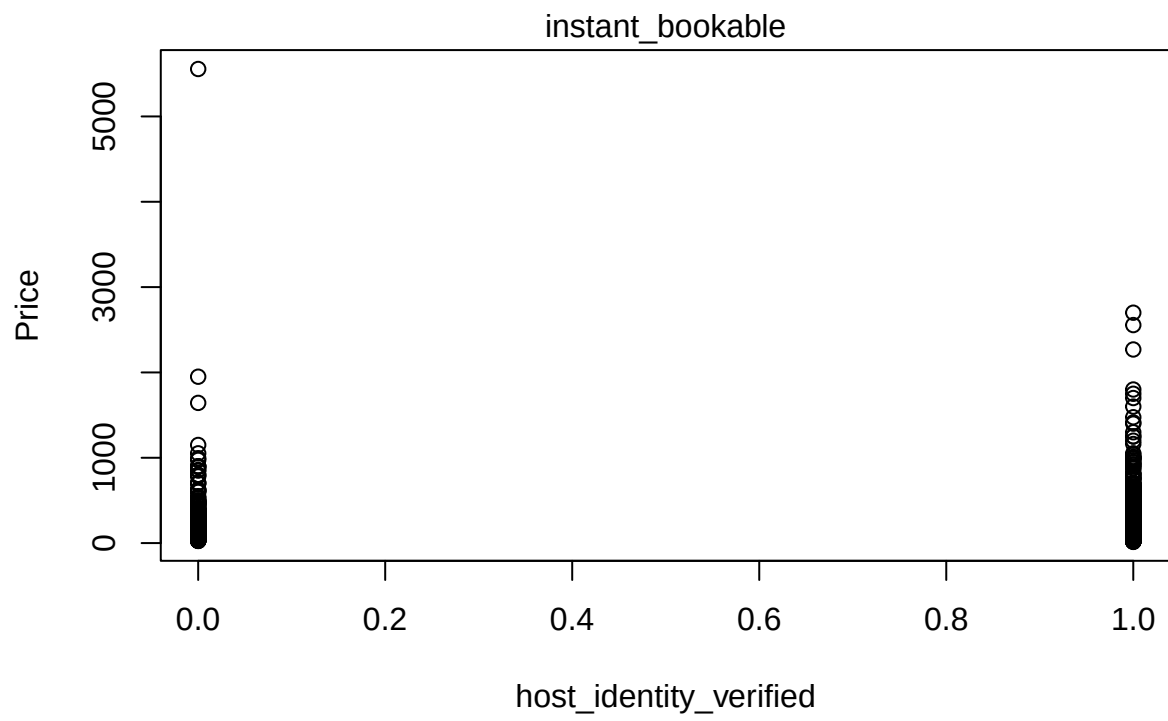
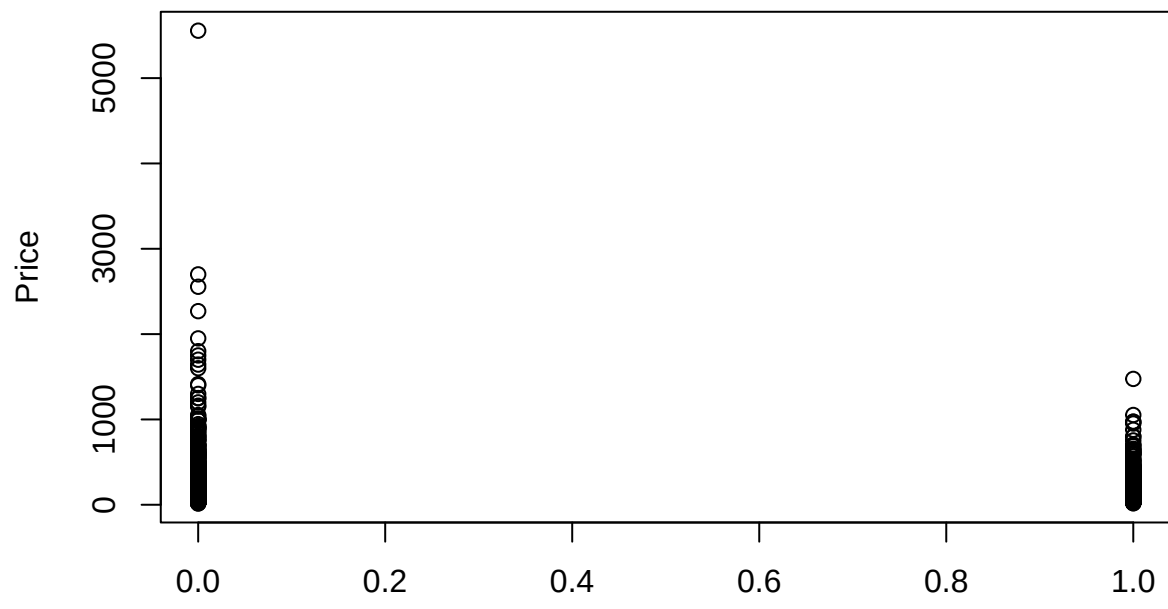


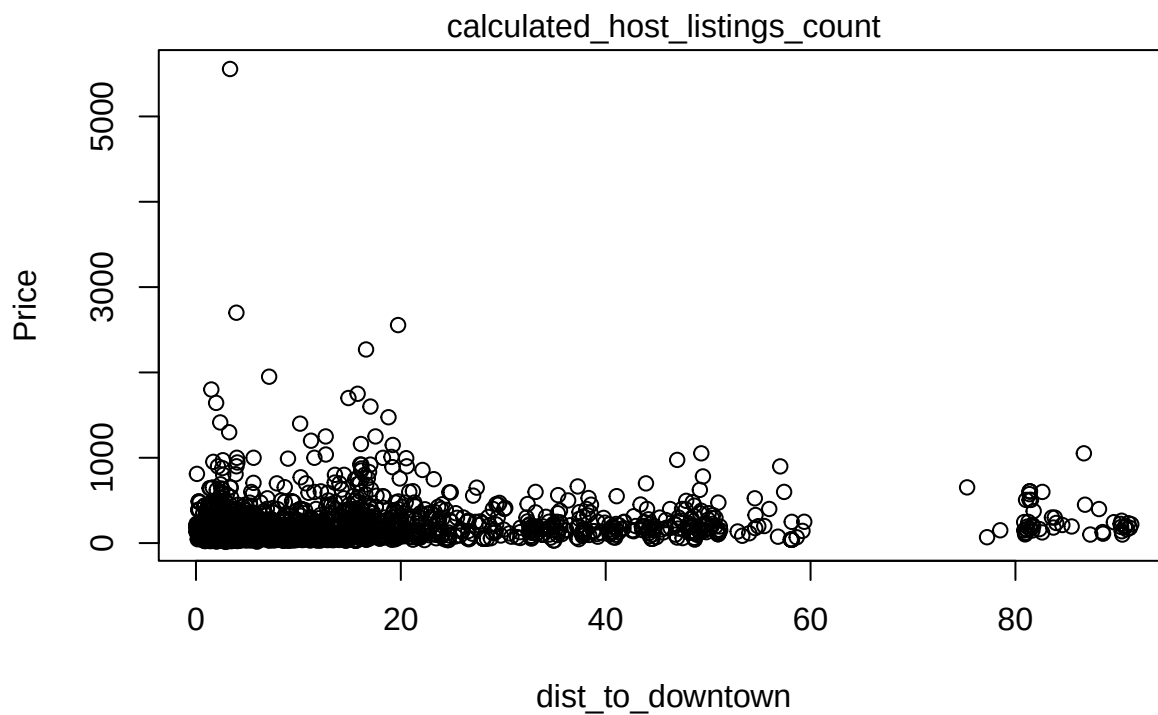
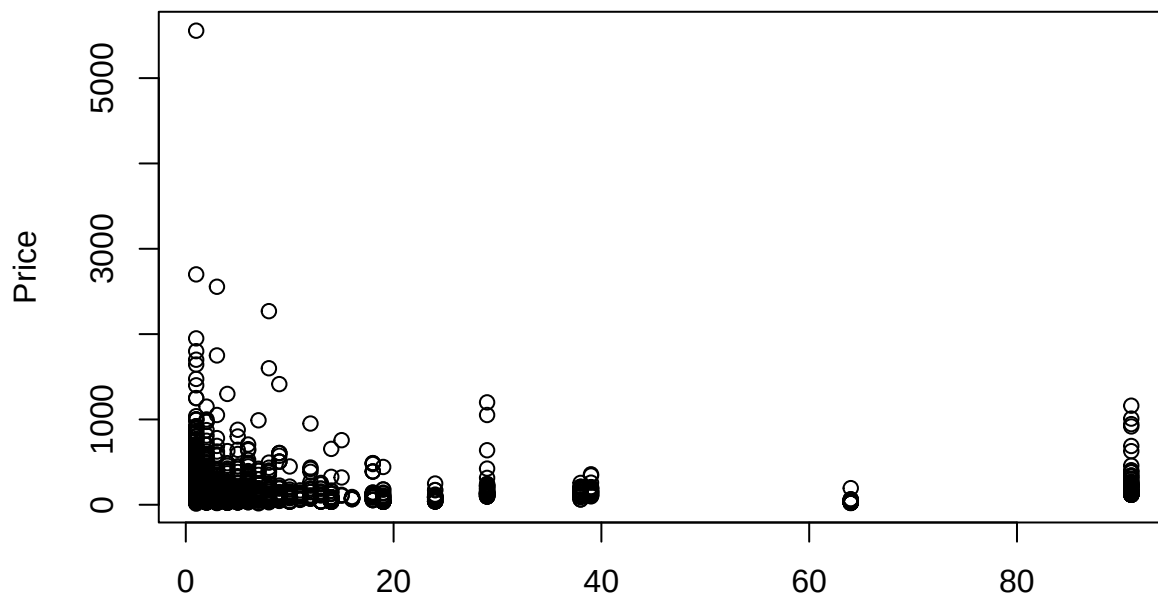












#Multivariate Analysis

#Correlation between pairs of variables

`cor(train.numeric)`

```
##               neighbourhood_cleansed  room_type  bathrooms
## neighbourhood_cleansed              1.000000000 -0.021281507  0.005958624
## room_type                        -0.0212815072  1.000000000 -0.151717384
## bathrooms                        0.0059586241 -0.151717384  1.000000000
## bedrooms                        0.0201013755 -0.318819257  0.690039920
## accommodates                     0.0571220657 -0.458702902  0.639449273
## minimum_nights                   -0.0750123153 -0.030701603 -0.051264594
```

## availability_365	0.0627764691	0.008513649	0.103218290
## number_of_reviews	-0.0278003837	-0.004146698	-0.146220262
## review_scores_rating	0.0116038997	-0.034159766	0.049365355
## review_scores_cleanliness	-0.0068075164	-0.018196438	-0.009849075
## review_scores_location	0.0190734482	-0.044550000	0.043017117
## host_is_superhost	0.0199507272	-0.029023081	-0.020035789
## instant_bookable	-0.0004803265	-0.007691496	0.041542380
## host_identity_verified	-0.0460143923	0.067154461	-0.018792745
## calculated_host_listings_count	-0.0497514668	0.009948699	0.104258860
## dist_to_downtown	0.2483382138	-0.066678126	0.150031130
## price	0.0634116183	-0.226873153	0.541281497
##	bedrooms	accommodates	minimum_nights
## neighbourhood_cleansed	0.020101376	0.05712207	-0.0750123153
## room_type	-0.318819257	-0.45870290	-0.0307016031
## bathrooms	0.690039920	0.63944927	-0.0512645936
## bedrooms	1.000000000	0.76613591	-0.0421455227
## accommodates	0.766135914	1.000000000	-0.0678057054
## minimum_nights	-0.042145523	-0.06780571	1.0000000000
## availability_365	0.075565117	0.10094349	-0.0456851350
## number_of_reviews	-0.131034852	-0.09011942	-0.0983398158
## review_scores_rating	0.012784551	0.04660902	0.0014455390
## review_scores_cleanliness	-0.031500117	0.00595134	-0.0460140321
## review_scores_location	0.009341250	0.03776113	-0.0004959422
## host_is_superhost	-0.051843402	-0.01321081	-0.1006861070
## instant_bookable	0.089481643	0.05959826	-0.0593276679
## host_identity_verified	-0.019421715	-0.04636865	0.0479424925
## calculated_host_listings_count	0.009874363	-0.03064852	-0.0368373915
## dist_to_downtown	0.126521251	0.23298600	-0.0635305524
## price	0.434306455	0.49885024	-0.0383131666
##	availability_365	number_of_reviews	
## neighbourhood_cleansed	0.062776469	-0.027800384	
## room_type	0.008513649	-0.004146698	
## bathrooms	0.103218290	-0.146220262	
## bedrooms	0.075565117	-0.131034852	
## accommodates	0.100943487	-0.090119422	
## minimum_nights	-0.045685135	-0.098339816	
## availability_365	1.000000000	-0.064907477	
## number_of_reviews	-0.064907477	1.000000000	
## review_scores_rating	-0.049958320	0.070396872	
## review_scores_cleanliness	-0.017281362	0.074113191	
## review_scores_location	-0.045995987	0.066314099	
## host_is_superhost	0.017534242	0.194821315	
## instant_bookable	0.094341229	-0.006568902	
## host_identity_verified	-0.011259321	0.059963814	
## calculated_host_listings_count	0.196922069	-0.090615548	
## dist_to_downtown	0.081270424	-0.128015252	
## price	0.032675006	-0.114293581	
##	review_scores_rating	review_scores_cleanliness	
## neighbourhood_cleansed	0.011603900	-0.006807516	
## room_type	-0.034159766	-0.018196438	
## bathrooms	0.049365355	-0.009849075	
## bedrooms	0.012784551	-0.031500117	
## accommodates	0.046609024	0.005951340	
## minimum_nights	0.001445539	-0.046014032	

## availability_365	-0.049958320	-0.017281362
## number_of_reviews	0.070396872	0.074113191
## review_scores_rating	1.000000000	0.739550633
## review_scores_cleanliness	0.739550633	1.000000000
## review_scores_location	0.590151605	0.446195365
## host_is_superhost	0.233504408	0.218945333
## instant_bookable	-0.053672757	-0.042111177
## host_identity_verified	-0.022858394	-0.015491049
## calculated_host_listings_count	-0.061856660	-0.026115252
## dist_to_downtown	0.020845980	0.011994799
## price	0.061460172	0.037355000
##	review_scores_location	host_is_superhost
## neighbourhood_cleansed	0.0190734482	0.01995073
## room_type	-0.0445499998	-0.02902308
## bathrooms	0.0430171173	-0.02003579
## bedrooms	0.0093412500	-0.05184340
## accommodates	0.0377611321	-0.01321081
## minimum_nights	-0.0004959422	-0.10068611
## availability_365	-0.0459959870	0.01753424
## number_of_reviews	0.0663140992	0.19482132
## review_scores_rating	0.5901516046	0.23350441
## review_scores_cleanliness	0.4461953646	0.21894533
## review_scores_location	1.0000000000	0.13189411
## host_is_superhost	0.1318941055	1.00000000
## instant_bookable	-0.0326451564	-0.05602417
## host_identity_verified	-0.0235486046	0.07353174
## calculated_host_listings_count	-0.0590612461	0.12190256
## dist_to_downtown	0.0416933343	-0.02482907
## price	0.0594888701	-0.04862159
##	instant_bookable	host_identity_verified
## neighbourhood_cleansed	-0.0004803265	-0.04601439
## room_type	-0.0076914956	0.06715446
## bathrooms	0.0415423805	-0.01879274
## bedrooms	0.0894816432	-0.01942171
## accommodates	0.0595982604	-0.04636865
## minimum_nights	-0.0593276679	0.04794249
## availability_365	0.0943412290	-0.01125932
## number_of_reviews	-0.0065689024	0.05996381
## review_scores_rating	-0.0536727566	-0.02285839
## review_scores_cleanliness	-0.0421111766	-0.01549105
## review_scores_location	-0.0326451564	-0.02354860
## host_is_superhost	-0.0560241743	0.07353174
## instant_bookable	1.0000000000	-0.05450000
## host_identity_verified	-0.0544999954	1.00000000
## calculated_host_listings_count	-0.0344304575	0.09652699
## dist_to_downtown	0.0826776774	-0.13942773
## price	0.0042112868	-0.07073172
##	calculated_host_listings_count	dist_to_downtown
## neighbourhood_cleansed	-0.049751467	0.24833821
## room_type	0.009948699	-0.06667813
## bathrooms	0.104258860	0.15003113
## bedrooms	0.009874363	0.12652125
## accommodates	-0.030648517	0.23298600
## minimum_nights	-0.036837392	-0.06353055

```
## availability_365          0.196922069      0.08127042
## number_of_reviews        -0.090615548     -0.12801525
## review_scores_rating     -0.061856660      0.02084598
## review_scores_cleanliness -0.026115252      0.01199480
## review_scores_location   -0.059061246      0.04169333
## host_is_superhost         0.121902558     -0.02482907
## instant_bookable         -0.034430458      0.08267768
## host_identity_verified    0.096526986     -0.13942773
## calculated_host_listings_count 1.000000000     -0.07620435
## dist_to_downtown         -0.076204345      1.00000000
## price                     0.002835539      0.15572343
##
##                               price
## neighbourhood_cleansed    0.063411618
## room_type                 -0.226873153
## bathrooms                 0.541281497
## bedrooms                 0.434306455
## accommodates              0.498850241
## minimum_nights            -0.038313167
## availability_365          0.032675006
## number_of_reviews         -0.114293581
## review_scores_rating       0.061460172
## review_scores_cleanliness  0.037355000
## review_scores_location     0.059488870
## host_is_superhost         -0.048621587
## instant_bookable           0.004211287
## host_identity_verified     -0.070731724
## calculated_host_listings_count 0.002835539
## dist_to_downtown           0.155723427
## price                     1.000000000
```

#Check for potential collinearity (none observed)

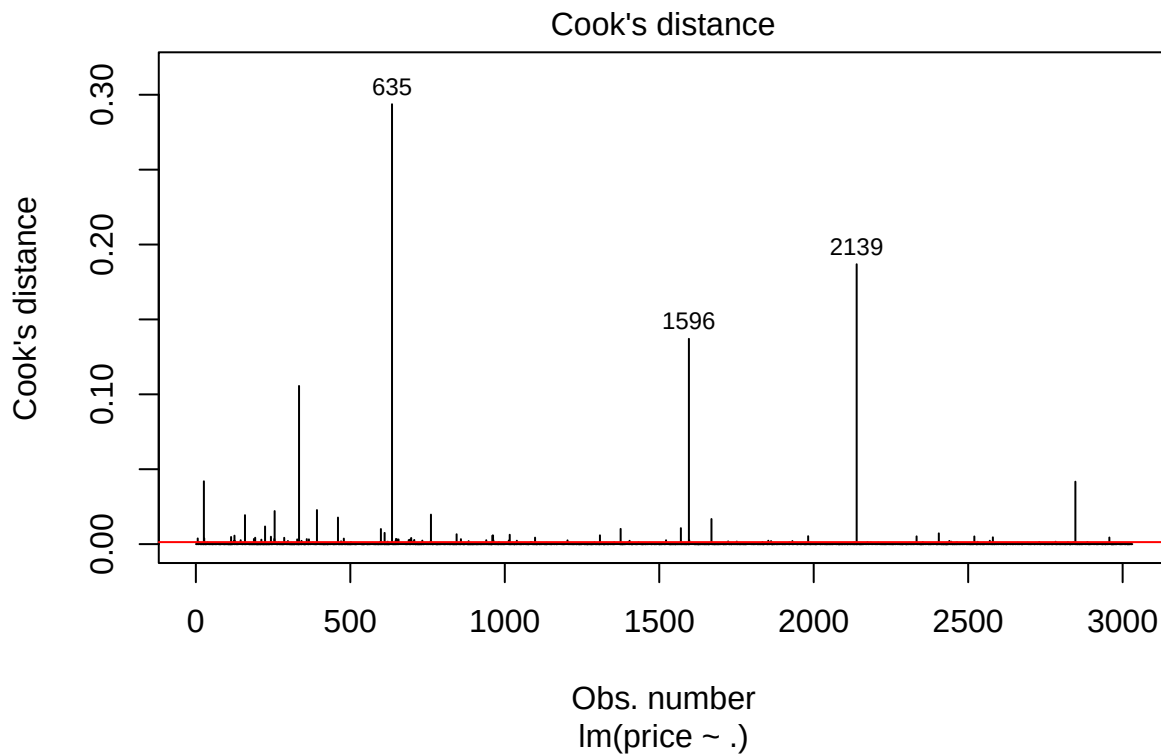
```
full_lm.numeric = lm(price ~ ., data = train.numeric)
print("VIF Values")
```

```
## [1] "VIF Values"
```

```
VIF(full_lm.numeric)
```

```
##      neighbourhood_cleansed      room_type
##      1.078296                1.350637
##      bathrooms              bedrooms
##      2.194217                2.971950
##      accommodates           minimum_nights
##      3.214697                1.051257
##      availability_365        number_of_reviews
##      1.080300                1.109928
##      review_scores_rating    review_scores_cleanliness
##      2.786389                2.245669
##      review_scores_location  host_is_superhost
##      1.543172                1.145593
##      instant_bookable        host_identity_verified
##      1.035009                1.044344
##      calculated_host_listings_count dist_to_downtown
##      1.129795                1.182169
```

```
#Influential points
plot(full_lm.numeric, which=4)
abline(h = 4/nrow(train.numeric), col="red")
```



```
cd = cooks.distance(full_lm.numeric)
influential.points = which(cd > 4/nrow(train.numeric))
length(influential.points)
```

```
## [1] 95
```

```
#Analyzing influential points
sort(cd, decreasing = T)[1:4]
```

```
##      635      2139      1596      334
## 0.2936891 0.1868347 0.1370516 0.1055825
```

```
train[635,]
```

```
##      neighbourhood_cleansed      room_type bathrooms bedrooms accommodates
## 635      Ramsey Private room              1              1              2
##      minimum_nights availability_365 number_of_reviews review_scores_rating
## 635              2              33              94              5
##      review_scores_cleanliness review_scores_location host_is_superhost
## 635              5              4.93              0
##      instant_bookable host_identity_verified calculated_host_listings_count
## 635              0              0              1
##      dist_to_downtown price
## 635      3.328816 5556
```

```
train[2139,]
```

```
##      neighbourhood_cleansed      room_type bathrooms bedrooms accommodates
```



```
## 2139          Anoka Entire home/apt          7.5          4          10
##      minimum_nights availability_365 number_of_reviews review_scores_rating
## 2139          2          365          4          5
##      review_scores_cleanliness review_scores_location host_is_superhost
## 2139          5          5          1
##      instant_bookable host_identity_verified calculated_host_listings_count
## 2139          0          1          3
##      dist_to_downtown price
## 2139          19.73122 2555
```

```
train[1596,]
```

```
##      neighbourhood_cleansed      room_type bathrooms bedrooms accommodates
## 1596      Hennepin Entire home/apt          5          4          10
##      minimum_nights availability_365 number_of_reviews review_scores_rating
## 1596          2          51          1          5
##      review_scores_cleanliness review_scores_location host_is_superhost
## 1596          4          5          0
##      instant_bookable host_identity_verified calculated_host_listings_count
## 1596          0          1          1
##      dist_to_downtown price
## 1596          3.946639 2700
```

```
train[334,]
```

```
##      neighbourhood_cleansed      room_type bathrooms bedrooms accommodates
## 334      Washington Entire home/apt          3.5          3          2
##      minimum_nights availability_365 number_of_reviews review_scores_rating
## 334          1          0          1          4
##      review_scores_cleanliness review_scores_location host_is_superhost
## 334          5          4          0
##      instant_bookable host_identity_verified calculated_host_listings_count
## 334          0          0          1
##      dist_to_downtown price
## 334          7.145394 1950
```

```
train.delete = train[-c(635, 2139, 1596, 334),]
```

```
train.delete.numeric = train.delete
```

```
train.delete.numeric$neighbourhood_cleansed = as.numeric(as.factor(train.delete.numeric$neighbourhood_cleansed))
```

```
train.delete.numeric$room_type = as.numeric(as.factor(train.delete.numeric$room_type))
```

```
##Model analysis with all predictors #Full linear model (base instance)
```

```
full_lm = lm(price ~ ., data = train)
full_lm.ypred = predict(full_lm, train)
full_lm.mse = mean((full_lm.ypred - train$price)^2)
sqrt(full_lm.mse)
```

```
## [1] 168.2705
```

```
summary(full_lm)
```

```
##
## Call:
## lm(formula = price ~ ., data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -396.7   -52.9    -9.2    29.2  5464.9
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -148.24233     61.57264   -2.408  0.016118 *
## neighbourhood_cleansedCarver      54.67941     29.09533    1.879  0.060298 .
## neighbourhood_cleansedChisago     13.09071     32.00571    0.409  0.682560
## neighbourhood_cleansedDakota    -16.53600     21.27699   -0.777  0.437115
## neighbourhood_cleansedHennepin    11.21084     16.82350    0.666  0.505220
## neighbourhood_cleansedIsanti    -20.14603     48.91193   -0.412  0.680454
## neighbourhood_cleansedLe Sueur     4.45507     52.06468    0.086  0.931816
## neighbourhood_cleansedMille Lacs  -76.19906     53.62638   -1.421  0.155443
## neighbourhood_cleansedPierce      20.89248     35.25070    0.593  0.553438
## neighbourhood_cleansedRamsey      13.69406     17.89161    0.765  0.444100
## neighbourhood_cleansedScott    -20.67744     36.99866   -0.559  0.576292
## neighbourhood_cleansedSherburne   -50.46589     35.26237   -1.431  0.152490
## neighbourhood_cleansedSibley    -54.66754     89.10726   -0.614  0.539591
## neighbourhood_cleansedSt. Croix    98.38929     30.34144    3.243  0.001197 **
## neighbourhood_cleansedWashington  71.14254     21.13359    3.366  0.000771 ***
## neighbourhood_cleansedWright    -24.32837     31.00260   -0.785  0.432680
## room_typeHotel room      41.72726     57.38872    0.727  0.467222
## room_typePrivate room    -33.76326      8.39082   -4.024  5.87e-05 ***
## room_typeShared room    125.76098    121.22206    1.037  0.299613
## bathrooms      98.53920      5.37824   18.322 < 2e-16 ***
## bedrooms     -7.24479      3.44275   -2.104  0.035430 *
## accommodates   14.07921      1.61780    8.703 < 2e-16 ***
## minimum_nights -0.09399      0.18969   -0.495  0.620315
## availability_365 -0.05559      0.02648   -2.099  0.035885 *
## number_of_reviews -0.10010      0.04288   -2.334  0.019643 *
## review_scores_rating -0.18211     15.83290   -0.012  0.990824
## review_scores_cleanliness  22.47099     13.92082    1.614  0.106589
## review_scores_location    7.35786     13.14280    0.560  0.575631
## host_is_superhost   -16.20682      6.62545   -2.446  0.014496 *
## instant_bookable   -11.49337      7.26008   -1.583  0.113506
## host_identity_verified -20.37448      8.12893   -2.506  0.012249 *
## calculated_host_listings_count -0.13804      0.21560   -0.640  0.522045
## dist_to_downtown     1.14911      0.65512    1.754  0.079528 .
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 169.2 on 2995 degrees of freedom
## Multiple R-squared:  0.3595, Adjusted R-squared:  0.3527
## F-statistic: 52.53 on 32 and 2995 DF,  p-value: < 2.2e-16

#Full linear model with influential points removed
full_lm.rem = lm(price ~ ., data = train.delete)
full_lm.rem.ypred = predict(full_lm.rem, train.delete)
full_lm.rem.mse = mean((full_lm.rem.ypred - train.delete$price)^2)
sqrt(full_lm.rem.mse)

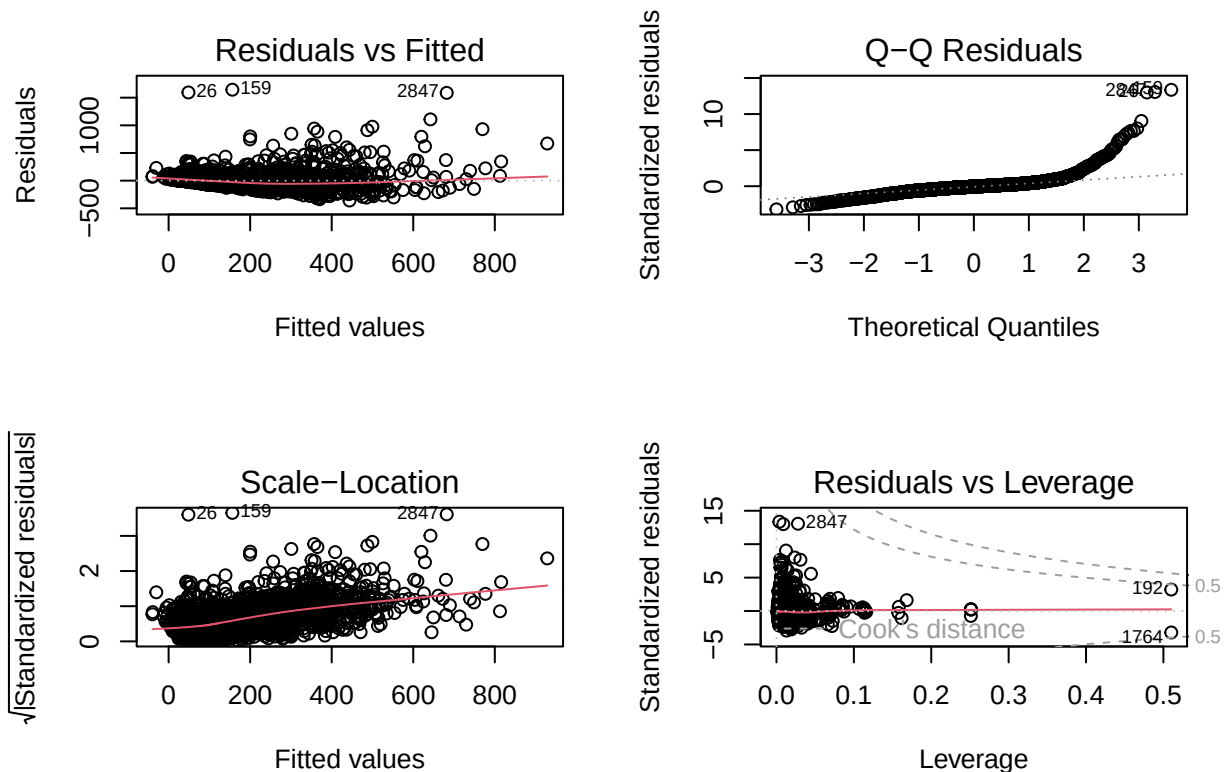
## [1] 122.5912

summary(full_lm.rem)

##
## Call:
## lm(formula = price ~ ., data = train.delete)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -358.69  -48.83   -7.09   28.15 1643.60
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -174.66593    44.91345   -3.889 0.000103 ***
## neighbourhood_cleansedCarver      71.74840    21.22736    3.380 0.000734 ***
## neighbourhood_cleansedChisago     16.70408    23.34808    0.715 0.474395
## neighbourhood_cleansedDakota     -4.27613    15.52822   -0.275 0.783045
## neighbourhood_cleansedHennepin    20.04678    12.28400    1.632 0.102797
## neighbourhood_cleansedIsanti    -19.42863    35.65627   -0.545 0.585872
## neighbourhood_cleansedLe Sueur     6.99936    37.95459    0.184 0.853701
## neighbourhood_cleansedMille Lacs -81.02371    39.11272   -2.072 0.038394 *
## neighbourhood_cleansedPierce      29.83809    25.71085    1.161 0.245928
## neighbourhood_cleansedRamsey      14.73043    13.06014    1.128 0.259456
## neighbourhood_cleansedScott      -2.38076    26.97886   -0.088 0.929688
## neighbourhood_cleansedSherburne  -47.62411    25.72011   -1.852 0.064178 .
## neighbourhood_cleansedSibley     -51.47188    64.92899   -0.793 0.427992
## neighbourhood_cleansedSt. Croix  111.86605    22.13635    5.053 4.60e-07 ***
## neighbourhood_cleansedWashington  70.90278    15.46104    4.586 4.71e-06 ***
## neighbourhood_cleansedWright    -22.74971    22.61737   -1.006 0.314568
## room_typeHotel room       37.49646    41.81209    0.897 0.369906
## room_typePrivate room    -34.49605     6.12366   -5.633 1.93e-08 ***
## room_typeShared room    124.50673    88.31762    1.410 0.158714
## bathrooms            80.56079     3.99093   20.186 < 2e-16 ***
## bedrooms            -3.33954     2.51440   -1.328 0.184225
## accommodates         15.39135     1.18147   13.027 < 2e-16 ***
## minimum_nights      -0.04625     0.13822   -0.335 0.737950
## availability_365     -0.02572     0.01932   -1.331 0.183140
## number_of_reviews    -0.11624     0.03124   -3.720 0.000203 ***
## review_scores_rating  -1.08172    11.57242   -0.093 0.925533
## review_scores_cleanliness 20.90010    10.19463    2.050 0.040441 *
## review_scores_location  11.31413     9.58219    1.181 0.237798
## host_is_superhost    -10.85714     4.82933   -2.248 0.024638 *
```

```
## instant_bookable          -7.02761    5.29028   -1.328  0.184147
## host_identity_verified    -8.79207    5.93282   -1.482  0.138462
## calculated_host_listings_count -0.03360    0.15720   -0.214  0.830770
## dist_to_downtown          1.34840    0.47758    2.823  0.004783 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 123.3 on 2991 degrees of freedom
## Multiple R-squared:  0.4925, Adjusted R-squared:  0.4871
## F-statistic: 90.71 on 32 and 2991 DF,  p-value: < 2.2e-16
```

```
par(mfrow=c(2,2))
plot(full_lm.rem)
```



```
#GLM with log link
```

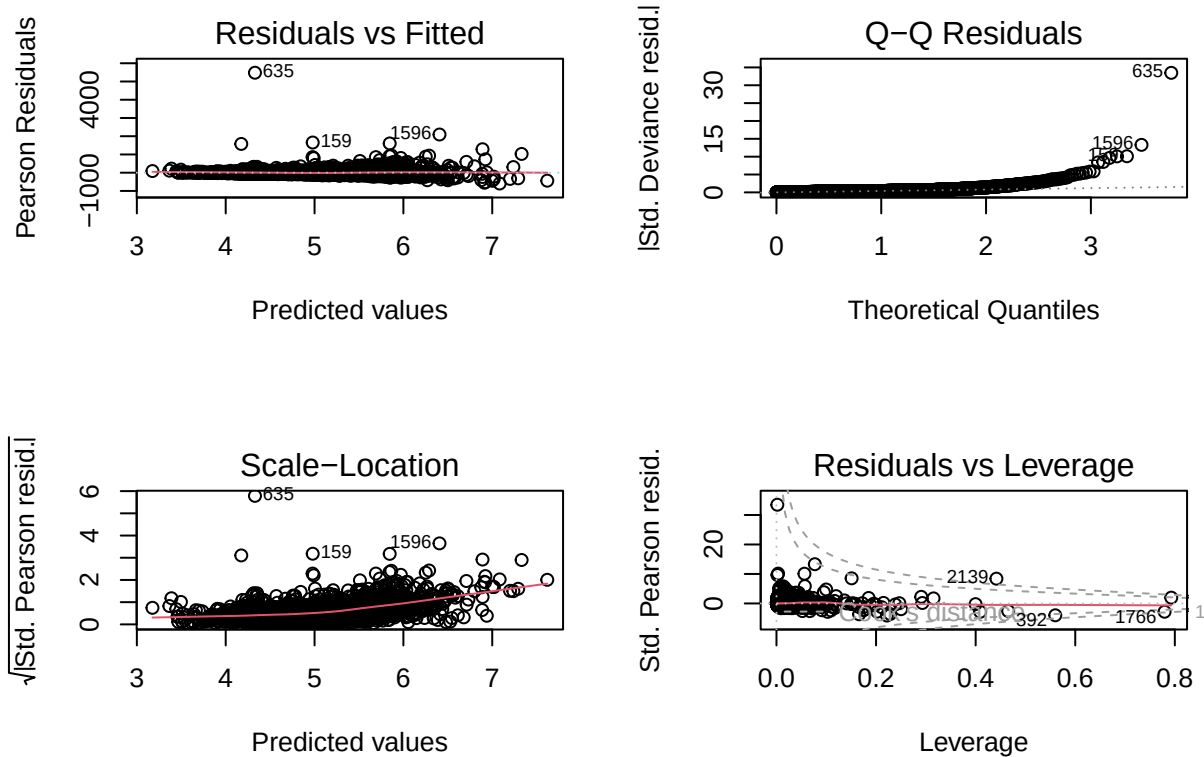
```
#Full dataset
log_glm = glm(price ~ ., data = train, family = gaussian(link="log"))
log_glm.ypred <- predict(log_glm, train, type = "response")
log_glm.mse = mean((log_glm.ypred - train$price)^2)
paste0("GLM with log link RMSE: ", round(sqrt(log_glm.mse),2))
```

```
## [1] "GLM with log link RMSE: 162.93"
```

```
#Deleted 4 influential points
log_glm4 = glm(price ~ ., data = train.delete, family = gaussian(link="log"))
log_glm.ypredm4 <- predict(log_glm, train.delete, type = "response")
log_glm.msem4 = mean((log_glm.ypredm4 - train.delete$price)^2)
paste0("GLM with log link RMSE (-4): ", round(sqrt(log_glm.msem4),2))
```

```
## [1] "GLM with log link RMSE (-4): 118.31"
```

```
par(mfrow=c(2,2))
plot(log_glm)
```



```
summary(log_glm)
```

```
##
## Call:
## glm(formula = price ~ ., family = gaussian(link = "log"), data = train)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      3.422e+00  3.864e-01   8.856 < 2e-16 ***
## neighbourhood_cleansedCarver  -3.395e-02  7.823e-02  -0.434  0.664328
## neighbourhood_cleansedChisago -2.173e-01  1.119e-01  -1.942  0.052174 .
## neighbourhood_cleansedDakota  -3.043e-01  9.241e-02  -3.293  0.001003 **
## neighbourhood_cleansedHennepin -1.732e-01  6.073e-02  -2.851  0.004385 **
## neighbourhood_cleansedIsanti  -4.797e-01  2.228e-01  -2.153  0.031377 *
## neighbourhood_cleansedLe Sueur  -8.007e-01  1.746e-01  -4.586  4.71e-06 ***
## neighbourhood_cleansedMille Lacs -7.006e-01  1.914e-01  -3.661  0.000256 ***
## neighbourhood_cleansedPierce   -2.469e-01  1.077e-01  -2.292  0.021956 *
## neighbourhood_cleansedRamsey   -1.702e-01  7.291e-02  -2.335  0.019616 *
## neighbourhood_cleansedScott    -2.700e-01  1.612e-01  -1.675  0.094080 .
## neighbourhood_cleansedSherburne -4.148e-01  1.684e-01  -2.463  0.013845 *
## neighbourhood_cleansedSibley   -7.167e-01  6.257e-01  -1.145  0.252105
## neighbourhood_cleansedSt. Croix  8.691e-03  7.700e-02   0.113  0.910138
## neighbourhood_cleansedWashington 1.035e-01  6.685e-02   1.549  0.121582
## neighbourhood_cleansedWright  -4.889e-01  1.205e-01  -4.056  5.13e-05 ***
## room_typeHotel room          1.107e-01  3.232e-01   0.342  0.732071
## room_typePrivate room       -5.039e-01  7.164e-02  -7.034  2.47e-12 ***
## room_typeShared room         6.342e-01  2.284e-01   2.777  0.005527 **
```

```
## bathrooms                2.672e-01  1.287e-02  20.761  < 2e-16 ***
## bedrooms                 2.240e-02  1.280e-02   1.750  0.080267 .
## accommodates             4.097e-02  5.971e-03   6.863  8.18e-12 ***
## minimum_nights          -4.195e-03  1.939e-03  -2.163  0.030588 *
## availability_365        -9.334e-05  1.285e-04  -0.726  0.467786
## number_of_reviews       -2.084e-03  3.796e-04  -5.491  4.33e-08 ***
## review_scores_rating     1.395e-01  9.399e-02   1.484  0.137870
## review_scores_cleanliness 7.005e-02  6.309e-02   1.110  0.266947
## review_scores_location   7.030e-02  7.937e-02   0.886  0.375869
## host_is_superhost       -6.808e-02  3.158e-02  -2.156  0.031190 *
## instant_bookable        -1.189e-01  3.545e-02  -3.354  0.000805 ***
## host_identity_verified   -1.970e-02  3.685e-02  -0.535  0.592958
## calculated_host_listings_count -2.920e-03  8.876e-04  -3.290  0.001014 **
## dist_to_downtown        7.754e-03  2.481e-03   3.125  0.001794 **
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
```

```
## (Dispersion parameter for gaussian family taken to be 26838.43)
```

```
##
```

```
## Null deviance: 133859761 on 3027 degrees of freedom
```

```
## Residual deviance: 80378852 on 2995 degrees of freedom
```

```
## AIC: 39506
```

```
##
```

```
## Number of Fisher Scoring iterations: 13
```

```
#Ridge and lasso regression (all predictors) w/CV
```

```
set.seed(1)
```

```
x = model.matrix(price ~ ., data = train)[-1]
```

```
y = train$price
```

```
#all data points
```

```
ridge.cv = cv.glmnet(x,y,alpha=0)
```

```
best.ridge.lambda = ridge.cv$lambda.min
```

```
ridge_model = glmnet(x, y, alpha=0, lambda = best.ridge.lambda)
```

```
ridge.ypred = predict(ridge_model, newx = x, lambda = best.ridge.lambda)
```

```
ridge.mse = mean((ridge.ypred - y)^2)
```

```
paste0("Full ridge RSME: ", round(sqrt(ridge.mse), 2))
```

```
## [1] "Full ridge RSME: 168.42"
```

```
lasso.cv = cv.glmnet(x,y,alpha=1)
```

```
best.lasso.lambda = lasso.cv$lambda.min
```

```
lasso_model = glmnet(x, y, alpha=1, lambda = best.lasso.lambda)
```

```
lasso.ypred = predict(lasso_model, newx = x, lambda = best.lasso.lambda)
```

```
lasso.mse = mean((lasso.ypred - y)^2)
```

```
paste0("Full lasso RSME: ", round(sqrt(lasso.mse), 2))
```

```
## [1] "Full lasso RSME: 168.41"
```

```
#deleted 4 influential points
```

```
xm4 = model.matrix(price ~ ., data = train)[-c(635, 2139, 1596, 334),-1]
```

```
ym4 = y[-c(635, 2139, 1596, 334)]
```

```
ridge.cvm4 = cv.glmnet(xm4,ym4,alpha=0)
```

```
best.ridge.lambdam4 = ridge.cvm4$lambda.min
```

```

ridge_modelm4 = glmnet(xm4, ym4, alpha=0, lambda = best.ridge.lambdam4)
ridge.ypredm4 = predict(ridge_modelm4, newx = xm4, lambda = best.ridge.lambdam4)
ridge.msem4 = mean((ridge.ypredm4 - ym4)^2)
paste0("Full ridge RSME (-4): ", round(sqrt(ridge.msem4), 2))

```

```
## [1] "Full ridge RSME (-4): 122.8"
```

```

lasso.cvm4 = cv.glmnet(xm4, ym4, alpha=1)
best.lasso.lambdam4 = lasso.cvm4$lambda.min
lasso_modelm4 = glmnet(xm4, ym4, alpha=1, lambda = best.lasso.lambdam4)
lasso.ypredm4 = predict(lasso_modelm4, newx = xm4, lambda = best.lasso.lambdam4)
lasso.msem4 = mean((lasso.ypredm4 - ym4)^2)
paste0("Full lasso RSME (-4): ", round(sqrt(lasso.msem4), 2))

```

```
## [1] "Full lasso RSME (-4): 122.92"
```

#Decision tree

```

set.seed(1)
#Full dataset
tree_model = tree(price ~ ., data = train.numeric)
tree_model.ypred = predict(tree_model, train.numeric)
tree.mse = mean((tree_model.ypred - train.numeric$price)^2)
paste0("Full tree RSME: ", round(sqrt(tree.mse), 2))

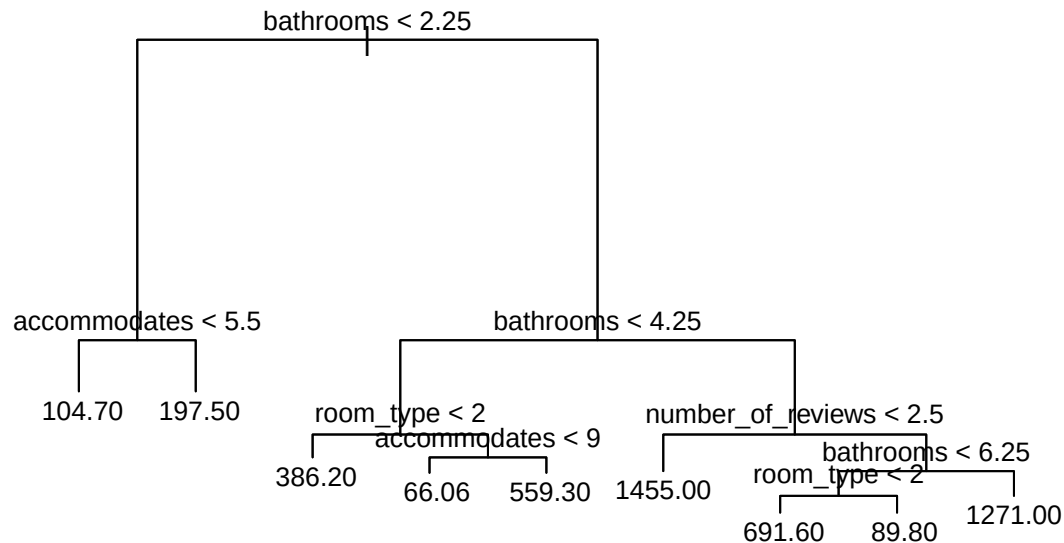
```

```
## [1] "Full tree RSME: 164.68"
```

```

plot(tree_model)
text(tree_model, cex=0.8)

```



```

#Deleted 4 influential points
tree_modelm4 = tree(price ~ ., data = train.delete.numeric)
tree_model.ypredm4 = predict(tree_modelm4, train.delete.numeric)
tree.msem4 = mean((tree_model.ypredm4 - train.delete.numeric$price)^2)
paste0("Full tree RSME (-4): ", round(sqrt(tree.msem4), 2))

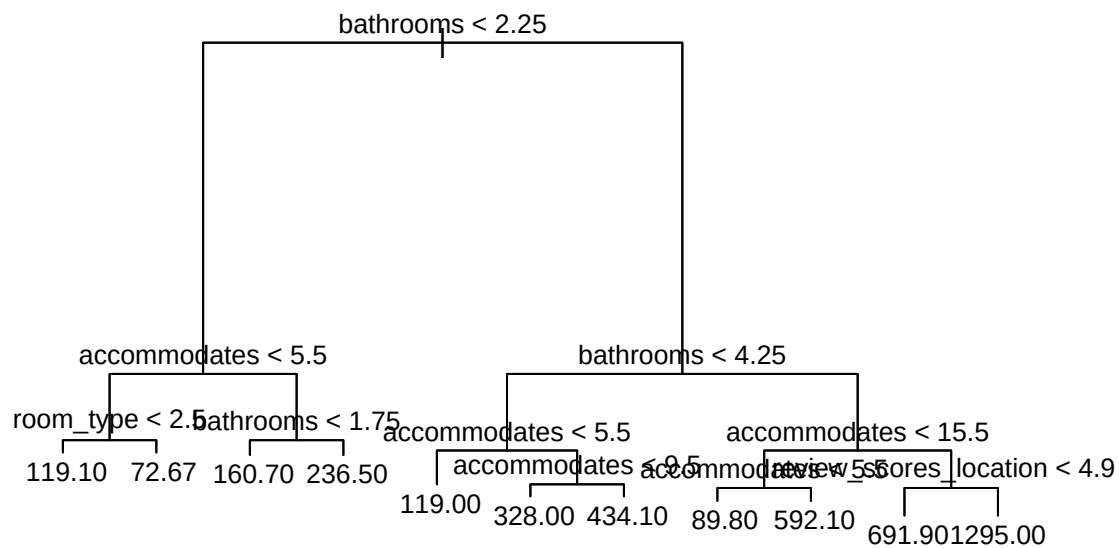
```

```
## [1] "Full tree RSME (-4): 118.23"
```

```

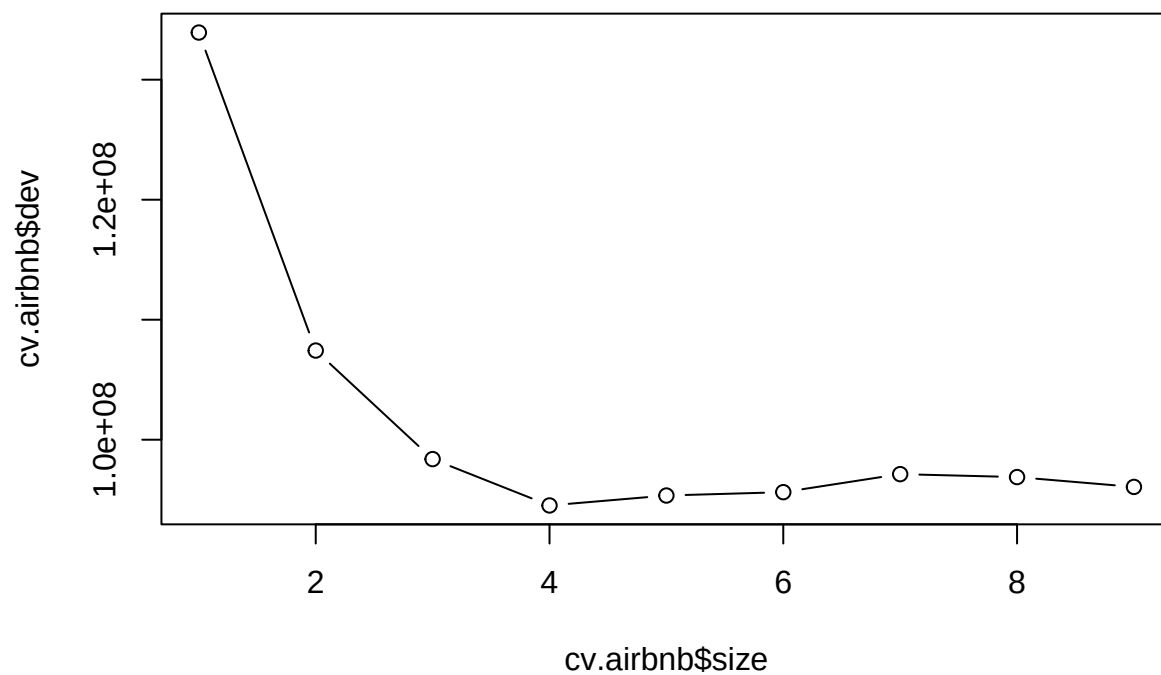
plot(tree_modelm4)
text(tree_modelm4, cex=0.8)

```



#Pruned tree with CV

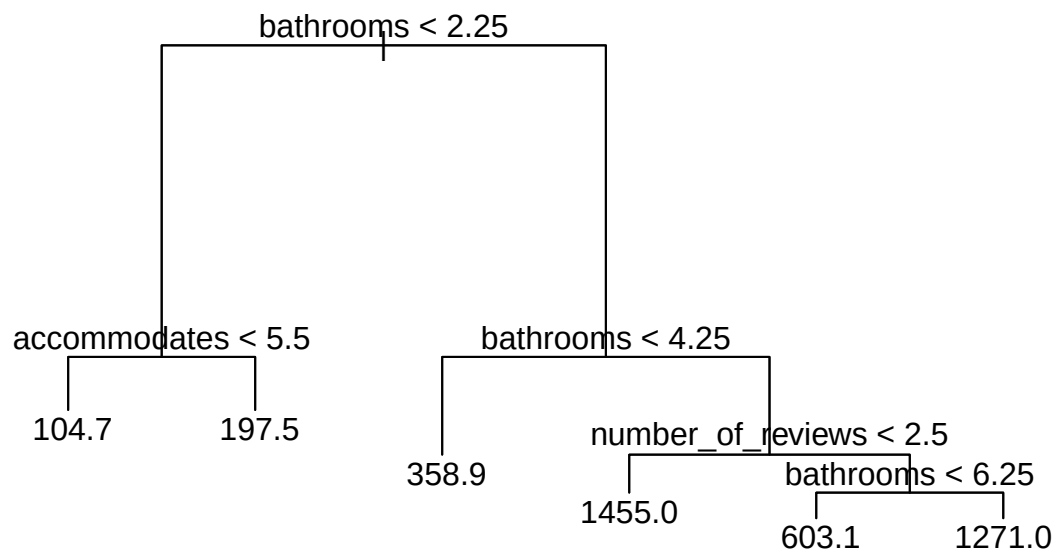
```
cv.airbnb = cv.tree(tree_model)
plot(cv.airbnb$size, cv.airbnb$dev, type = "b")
```



```
which.min(cv.airbnb$dev)
```

```
## [1] 6
```

```
pruned.tree_model = prune.tree(tree_model, best = 6)
plot(pruned.tree_model)
text(pruned.tree_model)
```

```
pruned.tree.ypred = predict(pruned.tree_model, train.numeric)
pruned.mse = mean((pruned.tree.ypred - train.numeric$price)^2)
paste0("Pruned tree RSME: ", round(sqrt(pruned.mse), 2))
```

```
## [1] "Pruned tree RSME: 169.68"
```

```
#Random forest
```

```
set.seed(1)
```

```
#Full dataset
```

```
rf_model = randomForest(price ~ ., data = train.numeric, importance=TRUE)
rf.ypred = predict(rf_model, train.numeric)
rf.mse = mean((rf.ypred - train.numeric$price)^2)
paste0("Random Forest RMSE: ", round(sqrt(rf.mse),2))
```

```
## [1] "Random Forest RMSE: 81.44"
```

```
print(rf_model)
```

```
##
```

```
## Call:
```

```
## randomForest(formula = price ~ ., data = train.numeric, importance = TRUE)
```

```
##           Type of random forest: regression
```

```
##           Number of trees: 500
```

```
## No. of variables tried at each split: 5
```

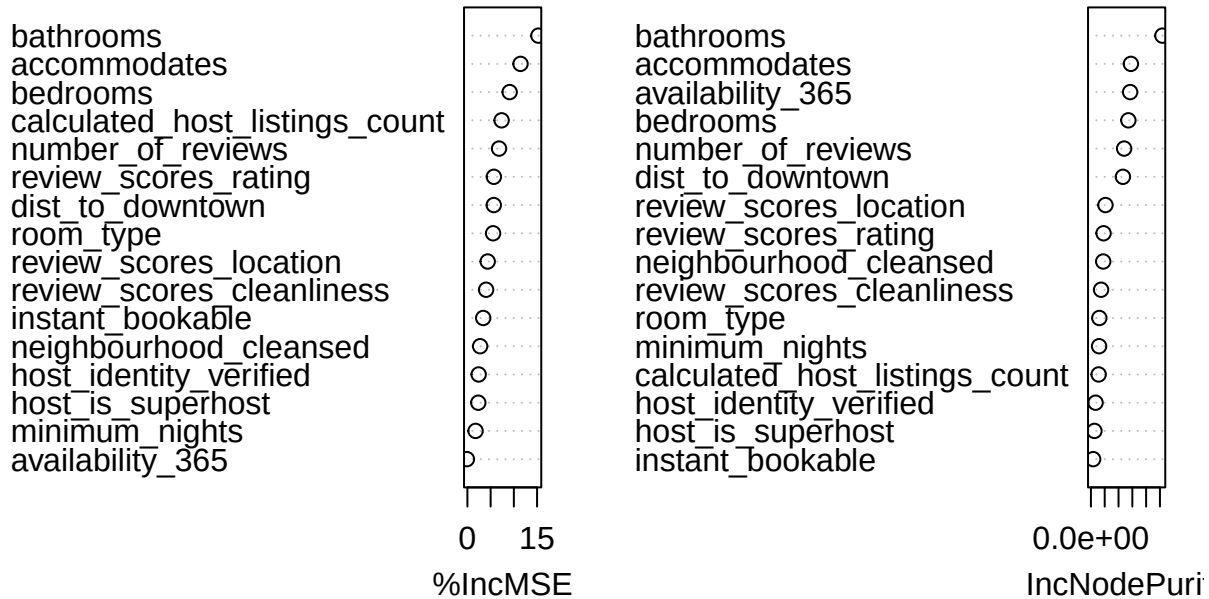
```
##
```

```
##           Mean of squared residuals: 27167.06
```

```
##           % Var explained: 38.55
```

```
varImpPlot(rf_model)
```

rf_model



```
importance(rf_model)
```

```
##
## %IncMSE IncNodePurity
## neighbourhood_cleansed 2.7442406 4457645.9
## room_type 5.5304745 3054888.1
## bathrooms 15.2876733 25896651.2
## bedrooms 9.1026272 13561024.2
## accommodates 11.4366512 14480241.7
## minimum_nights 1.7584537 2968388.2
## availability_365 -0.1053023 14216193.3
## number_of_reviews 6.8130656 12037969.9
## review_scores_rating 5.6948948 4578571.4
## review_scores_cleanliness 4.0320477 3678773.5
## review_scores_location 4.3715511 5264519.9
## host_is_superhost 2.3469155 1281097.1
## instant_bookable 3.4062127 818019.6
## host_identity_verified 2.4209092 1687773.6
## calculated_host_listings_count 7.3686903 2800807.1
## dist_to_downtown 5.6773246 11595395.5
```

```
#Deleted observations
```

```
rf_modelm4 = randomForest(price ~ ., data = train.delete.numeric, importance=TRUE)
rf.ypredm4 = predict(rf_modelm4, train.delete.numeric)
rf.msem4 = mean((rf.ypredm4 - train.delete.numeric$price)^2)
paste0("Random Forest RMSE (-4): ", round(sqrt(rf.msem4),2))
```

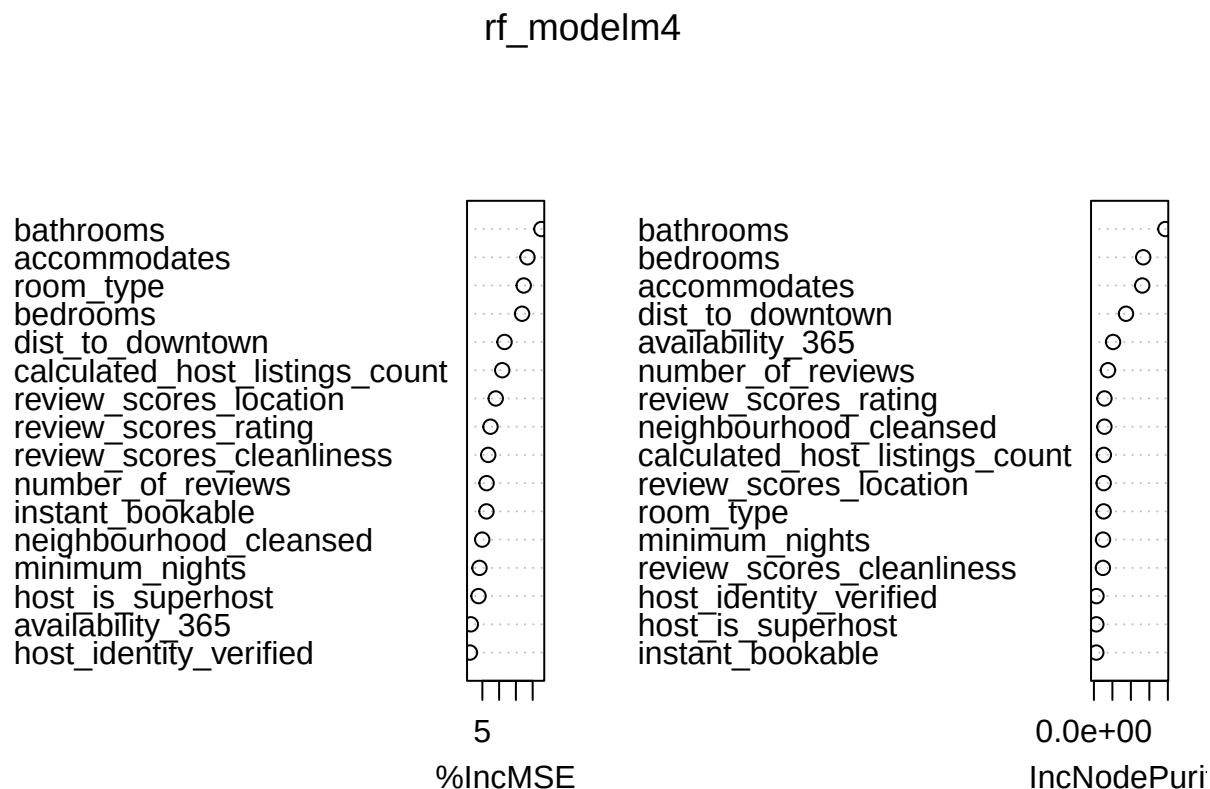
```
## [1] "Random Forest RMSE (-4): 56.01"
```

```
print(rf_modelm4)
```

```
##
```

```
## Call:
## randomForest(formula = price ~ ., data = train.delete.numeric, importance = TRUE)
##           Type of random forest: regression
##           Number of trees: 500
## No. of variables tried at each split: 5
##
##           Mean of squared residuals: 13406.19
##           % Var explained: 54.73
```

```
varImpPlot(rf_modelm4)
```



```
importance(rf_modelm4)
```

	%IncMSE	IncNodePurity
## neighbourhood_cleaned	4.949986	2834342.3
## room_type	17.332996	2631313.6
## bathrooms	22.608524	19338128.9
## bedrooms	16.848187	13337327.4
## accommodates	18.454624	13073283.4
## minimum_nights	4.140068	2471202.5
## availability_365	1.550915	5180127.7
## number_of_reviews	6.262998	3806093.6
## review_scores_rating	7.422449	2846672.2
## review_scores_cleanliness	6.755262	2429917.1
## review_scores_location	8.988995	2647679.8
## host_is_superhost	3.853021	648620.8
## instant_bookable	6.225221	642284.8
## host_identity_verified	1.284633	695708.1
## calculated_host_listings_count	10.929187	2690739.2
## dist_to_downtown	11.611017	8681849.4

```
##Model analysis with subset of predictors #Subset selection to obtain most significant variables
#Full dataset
regfit.forward = regsubsets(price ~ ., data = train, nvmax = 32, method = "forward")
reg.summary = summary(regfit.forward)
reg.summary

## Subset selection object
## Call: regsubsets.formula(price ~ ., data = train, nvmax = 32, method = "forward")
## 32 Variables (and intercept)
##
## Forced in Forced out
## neighbourhood_cleansedCarver FALSE FALSE
## neighbourhood_cleansedChisago FALSE FALSE
## neighbourhood_cleansedDakota FALSE FALSE
## neighbourhood_cleansedHennepin FALSE FALSE
## neighbourhood_cleansedIsanti FALSE FALSE
## neighbourhood_cleansedLe Sueur FALSE FALSE
## neighbourhood_cleansedMille Lacs FALSE FALSE
## neighbourhood_cleansedPierce FALSE FALSE
## neighbourhood_cleansedRamsey FALSE FALSE
## neighbourhood_cleansedScott FALSE FALSE
## neighbourhood_cleansedSherburne FALSE FALSE
## neighbourhood_cleansedSibley FALSE FALSE
## neighbourhood_cleansedSt. Croix FALSE FALSE
## neighbourhood_cleansedWashington FALSE FALSE
## neighbourhood_cleansedWright FALSE FALSE
## room_typeHotel room FALSE FALSE
## room_typePrivate room FALSE FALSE
## room_typeShared room FALSE FALSE
## bathrooms FALSE FALSE
## bedrooms FALSE FALSE
## accommodates FALSE FALSE
## minimum_nights FALSE FALSE
## availability_365 FALSE FALSE
## number_of_reviews FALSE FALSE
## review_scores_rating FALSE FALSE
## review_scores_cleanliness FALSE FALSE
## review_scores_location FALSE FALSE
## host_is_superhost FALSE FALSE
## instant_bookable FALSE FALSE
## host_identity_verified FALSE FALSE
## calculated_host_listings_count FALSE FALSE
## dist_to_downtown FALSE FALSE
## 1 subsets of each size up to 32
## Selection Algorithm: forward
##
## neighbourhood_cleansedCarver neighbourhood_cleansedChisago
## 1 ( 1 ) " " " "
## 2 ( 1 ) " " " "
## 3 ( 1 ) " " " "
## 4 ( 1 ) " " " "
## 5 ( 1 ) " " " "
## 6 ( 1 ) " " " "
## 7 ( 1 ) "*" " "
## 8 ( 1 ) "*" " "
## 9 ( 1 ) "*" " "
```

## 10	(1)	"*"	" "
## 11	(1)	"*"	" "
## 12	(1)	"*"	" "
## 13	(1)	"*"	"*"
## 14	(1)	"*"	"*"
## 15	(1)	"*"	"*"
## 16	(1)	"*"	"*"
## 17	(1)	"*"	"*"
## 18	(1)	"*"	"*"
## 19	(1)	"*"	"*"
## 20	(1)	"*"	"*"
## 21	(1)	"*"	"*"
## 22	(1)	"*"	"*"
## 23	(1)	"*"	"*"
## 24	(1)	"*"	"*"
## 25	(1)	"*"	"*"
## 26	(1)	"*"	"*"
## 27	(1)	"*"	"*"
## 28	(1)	"*"	"*"
## 29	(1)	"*"	"*"
## 30	(1)	"*"	"*"
## 31	(1)	"*"	"*"
## 32	(1)	"*"	"*"
##		neighbourhood_cleansedDakota	neighbourhood_cleansedHennepin
## 1	(1)	" "	" "
## 2	(1)	" "	" "
## 3	(1)	" "	" "
## 4	(1)	" "	" "
## 5	(1)	" "	" "
## 6	(1)	" "	" "
## 7	(1)	" "	" "
## 8	(1)	" "	" "
## 9	(1)	" "	" "
## 10	(1)	" "	" "
## 11	(1)	" "	" "
## 12	(1)	" "	" "
## 13	(1)	" "	" "
## 14	(1)	" "	" "
## 15	(1)	" "	" "
## 16	(1)	"*"	" "
## 17	(1)	"*"	" "
## 18	(1)	"*"	" "
## 19	(1)	"*"	" "
## 20	(1)	"*"	" "
## 21	(1)	"*"	" "
## 22	(1)	"*"	" "
## 23	(1)	"*"	" "
## 24	(1)	"*"	" "
## 25	(1)	"*"	" "
## 26	(1)	"*"	" "
## 27	(1)	"*"	" "
## 28	(1)	"*"	" "
## 29	(1)	"*"	" "
## 30	(1)	"*"	" "

```

## 31 ( 1 ) "*" "*"
## 32 ( 1 ) "*" "*"
##      neighbourhood_cleansedIsanti neighbourhood_cleansedLe Sueur
## 1 ( 1 ) " " " "
## 2 ( 1 ) " " " "
## 3 ( 1 ) " " " "
## 4 ( 1 ) " " " "
## 5 ( 1 ) " " " "
## 6 ( 1 ) " " " "
## 7 ( 1 ) " " " "
## 8 ( 1 ) " " " "
## 9 ( 1 ) " " " "
## 10 ( 1 ) " " " "
## 11 ( 1 ) " " " "
## 12 ( 1 ) " " " "
## 13 ( 1 ) " " " "
## 14 ( 1 ) " " " "
## 15 ( 1 ) " " " "
## 16 ( 1 ) " " " "
## 17 ( 1 ) " " " "
## 18 ( 1 ) " " "*"
## 19 ( 1 ) " " "*"
## 20 ( 1 ) " " "*"
## 21 ( 1 ) " " "*"
## 22 ( 1 ) " " "*"
## 23 ( 1 ) " " "*"
## 24 ( 1 ) " " "*"
## 25 ( 1 ) " " "*"
## 26 ( 1 ) " " "*"
## 27 ( 1 ) " " "*"
## 28 ( 1 ) "*" "*"
## 29 ( 1 ) "*" "*"
## 30 ( 1 ) "*" "*"
## 31 ( 1 ) "*" "*"
## 32 ( 1 ) "*" "*"
##      neighbourhood_cleansedMille Lacs neighbourhood_cleansedPierce
## 1 ( 1 ) " " " "
## 2 ( 1 ) " " " "
## 3 ( 1 ) " " " "
## 4 ( 1 ) " " " "
## 5 ( 1 ) " " " "
## 6 ( 1 ) " " " "
## 7 ( 1 ) " " " "
## 8 ( 1 ) " " " "
## 9 ( 1 ) " " " "
## 10 ( 1 ) " " " "
## 11 ( 1 ) " " " "
## 12 ( 1 ) " " " "
## 13 ( 1 ) " " " "
## 14 ( 1 ) " " " "
## 15 ( 1 ) " " "*"
## 16 ( 1 ) " " "*"
## 17 ( 1 ) " " "*"
## 18 ( 1 ) " " "*"

```

```

## 19 ( 1 ) " " "*"
## 20 ( 1 ) " " "*"
## 21 ( 1 ) " " "*"
## 22 ( 1 ) " " "*"
## 23 ( 1 ) "*" "*"
## 24 ( 1 ) "*" "*"
## 25 ( 1 ) "*" "*"
## 26 ( 1 ) "*" "*"
## 27 ( 1 ) "*" "*"
## 28 ( 1 ) "*" "*"
## 29 ( 1 ) "*" "*"
## 30 ( 1 ) "*" "*"
## 31 ( 1 ) "*" "*"
## 32 ( 1 ) "*" "*"
##      neighbourhood_cleansedRamsey neighbourhood_cleansedScott
## 1 ( 1 ) " " " "
## 2 ( 1 ) " " " "
## 3 ( 1 ) " " " "
## 4 ( 1 ) " " " "
## 5 ( 1 ) " " " "
## 6 ( 1 ) " " " "
## 7 ( 1 ) " " " "
## 8 ( 1 ) " " " "
## 9 ( 1 ) " " " "
## 10 ( 1 ) " " " "
## 11 ( 1 ) " " " "
## 12 ( 1 ) " " " "
## 13 ( 1 ) " " " "
## 14 ( 1 ) " " " "
## 15 ( 1 ) " " " "
## 16 ( 1 ) " " " "
## 17 ( 1 ) " " " "
## 18 ( 1 ) " " " "
## 19 ( 1 ) " " " "
## 20 ( 1 ) " " " "
## 21 ( 1 ) " " " "
## 22 ( 1 ) " " " "
## 23 ( 1 ) " " " "
## 24 ( 1 ) " " " "
## 25 ( 1 ) " " "*"
## 26 ( 1 ) " " "*"
## 27 ( 1 ) " " "*"
## 28 ( 1 ) " " "*"
## 29 ( 1 ) " " "*"
## 30 ( 1 ) "*" "*"
## 31 ( 1 ) "*" "*"
## 32 ( 1 ) "*" "*"
##      neighbourhood_cleansedSherburne neighbourhood_cleansedSibley
## 1 ( 1 ) " " " "
## 2 ( 1 ) " " " "
## 3 ( 1 ) " " " "
## 4 ( 1 ) " " " "
## 5 ( 1 ) " " " "
## 6 ( 1 ) " " " "

```

## 7	(1)	" "	" "
## 8	(1)	" "	" "
## 9	(1)	" "	" "
## 10	(1)	" "	" "
## 11	(1)	" "	" "
## 12	(1)	" "	" "
## 13	(1)	" "	" "
## 14	(1)	" "	" "
## 15	(1)	" "	" "
## 16	(1)	" "	" "
## 17	(1)	" "	" "
## 18	(1)	" "	" "
## 19	(1)	" "	" "
## 20	(1)	" "	" "
## 21	(1)	"*"	" "
## 22	(1)	"*"	" "
## 23	(1)	"*"	" "
## 24	(1)	"*"	" "
## 25	(1)	"*"	" "
## 26	(1)	"*"	"*"
## 27	(1)	"*"	"*"
## 28	(1)	"*"	"*"
## 29	(1)	"*"	"*"
## 30	(1)	"*"	"*"
## 31	(1)	"*"	"*"
## 32	(1)	"*"	"*"
##		neighbourhood_cleansedSt. Croix	neighbourhood_cleansedWashington
## 1	(1)	" "	" "
## 2	(1)	" "	" "
## 3	(1)	" "	"*"
## 4	(1)	" "	"*"
## 5	(1)	"*"	"*"
## 6	(1)	"*"	"*"
## 7	(1)	"*"	"*"
## 8	(1)	"*"	"*"
## 9	(1)	"*"	"*"
## 10	(1)	"*"	"*"
## 11	(1)	"*"	"*"
## 12	(1)	"*"	"*"
## 13	(1)	"*"	"*"
## 14	(1)	"*"	"*"
## 15	(1)	"*"	"*"
## 16	(1)	"*"	"*"
## 17	(1)	"*"	"*"
## 18	(1)	"*"	"*"
## 19	(1)	"*"	"*"
## 20	(1)	"*"	"*"
## 21	(1)	"*"	"*"
## 22	(1)	"*"	"*"
## 23	(1)	"*"	"*"
## 24	(1)	"*"	"*"
## 25	(1)	"*"	"*"
## 26	(1)	"*"	"*"
## 27	(1)	"*"	"*"


```

## 28 ( 1 ) "*"                "*"
## 29 ( 1 ) "*"                "*"
## 30 ( 1 ) "*"                "*"
## 31 ( 1 ) "*"                "*"
## 32 ( 1 ) "*"                "*"
##      neighbourhood_cleansedWright room_typeHotel room
## 1 ( 1 ) " "                  " "
## 2 ( 1 ) " "                  " "
## 3 ( 1 ) " "                  " "
## 4 ( 1 ) " "                  " "
## 5 ( 1 ) " "                  " "
## 6 ( 1 ) " "                  " "
## 7 ( 1 ) " "                  " "
## 8 ( 1 ) " "                  " "
## 9 ( 1 ) " "                  " "
## 10 ( 1 ) " "                 " "
## 11 ( 1 ) " "                 " "
## 12 ( 1 ) " "                 " "
## 13 ( 1 ) " "                 " "
## 14 ( 1 ) " "                 " "
## 15 ( 1 ) " "                 " "
## 16 ( 1 ) " "                 " "
## 17 ( 1 ) " "                 " "
## 18 ( 1 ) " "                 " "
## 19 ( 1 ) " "                 " "
## 20 ( 1 ) " "                 "*"
## 21 ( 1 ) " "                 "*"
## 22 ( 1 ) " "                 "*"
## 23 ( 1 ) " "                 "*"
## 24 ( 1 ) "*"                 "*"
## 25 ( 1 ) "*"                 "*"
## 26 ( 1 ) "*"                 "*"
## 27 ( 1 ) "*"                 "*"
## 28 ( 1 ) "*"                 "*"
## 29 ( 1 ) "*"                 "*"
## 30 ( 1 ) "*"                 "*"
## 31 ( 1 ) "*"                 "*"
## 32 ( 1 ) "*"                 "*"
##      room_typePrivate room room_typeShared room bathrooms bedrooms
## 1 ( 1 ) " "                  " "                "*"          " "
## 2 ( 1 ) " "                  " "                "*"          " "
## 3 ( 1 ) " "                  " "                "*"          " "
## 4 ( 1 ) "*"                  " "                "*"          " "
## 5 ( 1 ) "*"                  " "                "*"          " "
## 6 ( 1 ) "*"                  " "                "*"          " "
## 7 ( 1 ) "*"                  " "                "*"          " "
## 8 ( 1 ) "*"                  " "                "*"          " "
## 9 ( 1 ) "*"                  " "                "*"          " "
## 10 ( 1 ) "*"                  " "                "*"          " "
## 11 ( 1 ) "*"                  " "                "*"          "*"
## 12 ( 1 ) "*"                  " "                "*"          "*"
## 13 ( 1 ) "*"                  " "                "*"          "*"
## 14 ( 1 ) "*"                  " "                "*"          "*"
## 15 ( 1 ) "*"                  " "                "*"          "*"

```

## 16	(1)	"*"	" "	"*"	"*"
## 17	(1)	"*"	"*"	"*"	"*"
## 18	(1)	"*"	"*"	"*"	"*"
## 19	(1)	"*"	"*"	"*"	"*"
## 20	(1)	"*"	"*"	"*"	"*"
## 21	(1)	"*"	"*"	"*"	"*"
## 22	(1)	"*"	"*"	"*"	"*"
## 23	(1)	"*"	"*"	"*"	"*"
## 24	(1)	"*"	"*"	"*"	"*"
## 25	(1)	"*"	"*"	"*"	"*"
## 26	(1)	"*"	"*"	"*"	"*"
## 27	(1)	"*"	"*"	"*"	"*"
## 28	(1)	"*"	"*"	"*"	"*"
## 29	(1)	"*"	"*"	"*"	"*"
## 30	(1)	"*"	"*"	"*"	"*"
## 31	(1)	"*"	"*"	"*"	"*"
## 32	(1)	"*"	"*"	"*"	"*"
##		accommodates	minimum_nights	availability_365	number_of_reviews
## 1	(1)	" "	" "	" "	" "
## 2	(1)	"*"	" "	" "	" "
## 3	(1)	"*"	" "	" "	" "
## 4	(1)	"*"	" "	" "	" "
## 5	(1)	"*"	" "	" "	" "
## 6	(1)	"*"	" "	" "	" "
## 7	(1)	"*"	" "	" "	" "
## 8	(1)	"*"	" "	" "	"*"
## 9	(1)	"*"	" "	" "	"*"
## 10	(1)	"*"	" "	" "	"*"
## 11	(1)	"*"	" "	" "	"*"
## 12	(1)	"*"	" "	"*"	"*"
## 13	(1)	"*"	" "	"*"	"*"
## 14	(1)	"*"	" "	"*"	"*"
## 15	(1)	"*"	" "	"*"	"*"
## 16	(1)	"*"	" "	"*"	"*"
## 17	(1)	"*"	" "	"*"	"*"
## 18	(1)	"*"	" "	"*"	"*"
## 19	(1)	"*"	" "	"*"	"*"
## 20	(1)	"*"	" "	"*"	"*"
## 21	(1)	"*"	" "	"*"	"*"
## 22	(1)	"*"	" "	"*"	"*"
## 23	(1)	"*"	" "	"*"	"*"
## 24	(1)	"*"	" "	"*"	"*"
## 25	(1)	"*"	" "	"*"	"*"
## 26	(1)	"*"	" "	"*"	"*"
## 27	(1)	"*"	" "	"*"	"*"
## 28	(1)	"*"	" "	"*"	"*"
## 29	(1)	"*"	"*"	"*"	"*"
## 30	(1)	"*"	"*"	"*"	"*"
## 31	(1)	"*"	"*"	"*"	"*"
## 32	(1)	"*"	"*"	"*"	"*"
##		review_scores_rating	review_scores_cleanliness	review_scores_location	
## 1	(1)	" "	" "	" "	
## 2	(1)	" "	" "	" "	
## 3	(1)	" "	" "	" "	

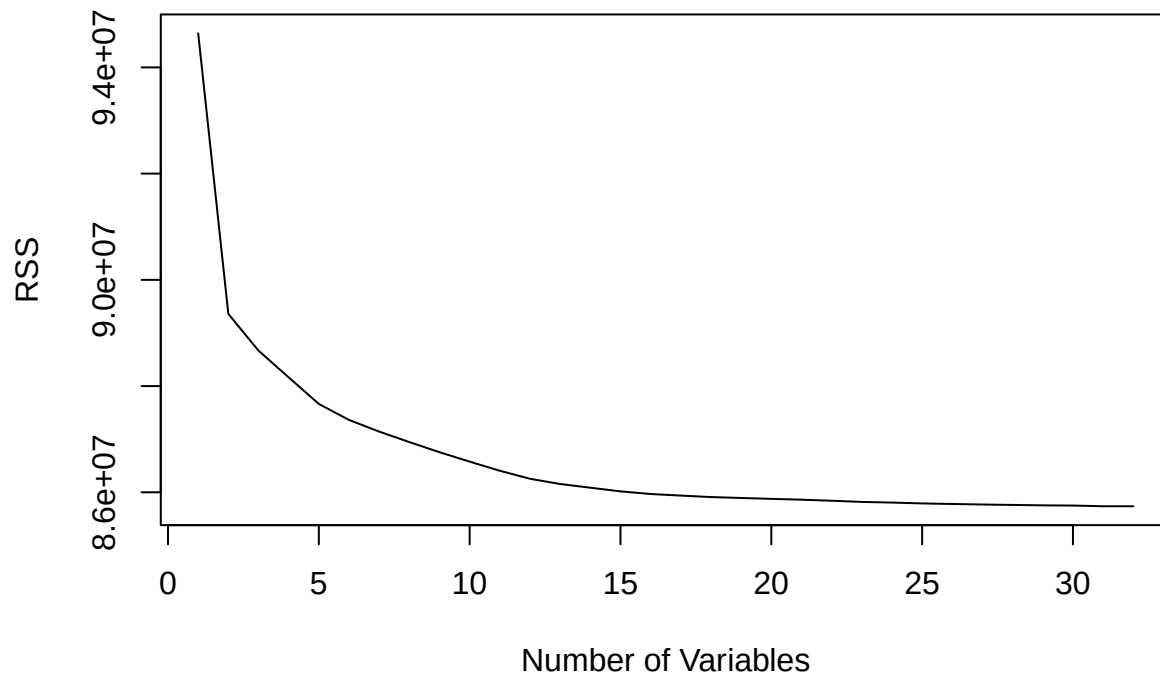
## 4	(1)	" "	" "	" "
## 5	(1)	" "	" "	" "
## 6	(1)	" "	" "	" "
## 7	(1)	" "	" "	" "
## 8	(1)	" "	" "	" "
## 9	(1)	" "	"*"	" "
## 10	(1)	" "	"*"	" "
## 11	(1)	" "	"*"	" "
## 12	(1)	" "	"*"	" "
## 13	(1)	" "	"*"	" "
## 14	(1)	" "	"*"	" "
## 15	(1)	" "	"*"	" "
## 16	(1)	" "	"*"	" "
## 17	(1)	" "	"*"	" "
## 18	(1)	" "	"*"	" "
## 19	(1)	" "	"*"	"*"
## 20	(1)	" "	"*"	"*"
## 21	(1)	" "	"*"	"*"
## 22	(1)	" "	"*"	"*"
## 23	(1)	" "	"*"	"*"
## 24	(1)	" "	"*"	"*"
## 25	(1)	" "	"*"	"*"
## 26	(1)	" "	"*"	"*"
## 27	(1)	" "	"*"	"*"
## 28	(1)	" "	"*"	"*"
## 29	(1)	" "	"*"	"*"
## 30	(1)	" "	"*"	"*"
## 31	(1)	" "	"*"	"*"
## 32	(1)	"*"	"*"	"*"
##		host_is_superuser	instant_bookable	host_identity_verified
## 1	(1)	" "	" "	" "
## 2	(1)	" "	" "	" "
## 3	(1)	" "	" "	" "
## 4	(1)	" "	" "	" "
## 5	(1)	" "	" "	" "
## 6	(1)	" "	" "	"*"
## 7	(1)	" "	" "	"*"
## 8	(1)	" "	" "	"*"
## 9	(1)	" "	" "	"*"
## 10	(1)	"*"	" "	"*"
## 11	(1)	"*"	" "	"*"
## 12	(1)	"*"	" "	"*"
## 13	(1)	"*"	" "	"*"
## 14	(1)	"*"	"*"	"*"
## 15	(1)	"*"	"*"	"*"
## 16	(1)	"*"	"*"	"*"
## 17	(1)	"*"	"*"	"*"
## 18	(1)	"*"	"*"	"*"
## 19	(1)	"*"	"*"	"*"
## 20	(1)	"*"	"*"	"*"
## 21	(1)	"*"	"*"	"*"
## 22	(1)	"*"	"*"	"*"
## 23	(1)	"*"	"*"	"*"
## 24	(1)	"*"	"*"	"*"

```

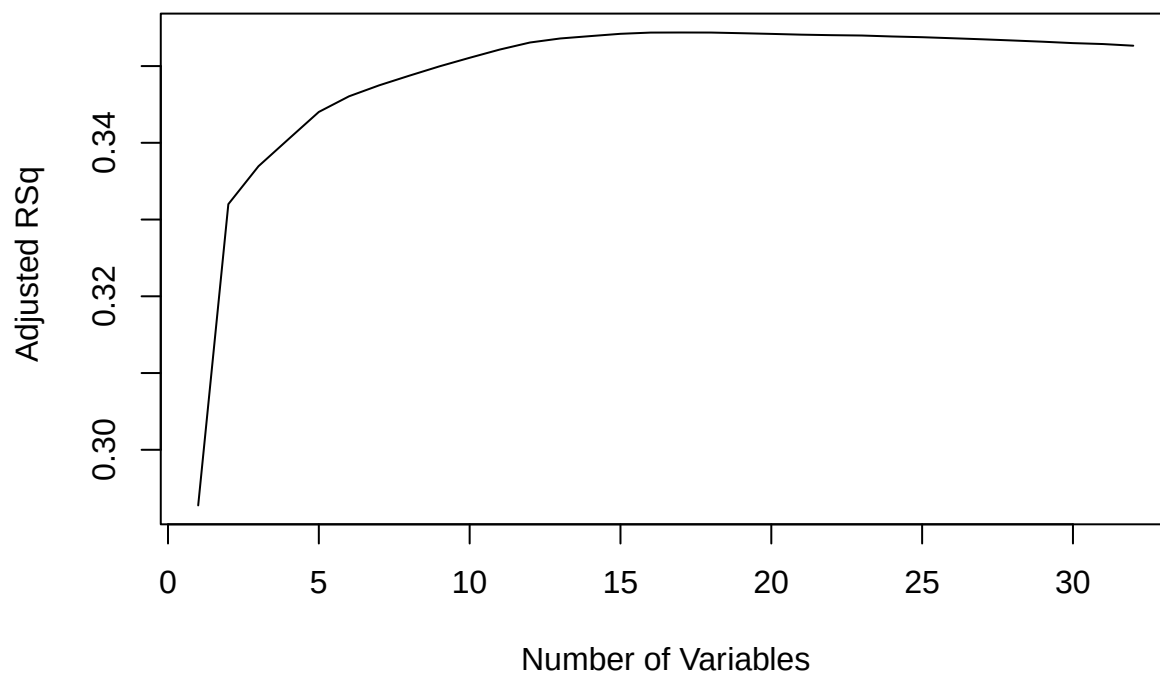
## 25 ( 1 ) "*"          "*"          "*"
## 26 ( 1 ) "*"          "*"          "*"
## 27 ( 1 ) "*"          "*"          "*"
## 28 ( 1 ) "*"          "*"          "*"
## 29 ( 1 ) "*"          "*"          "*"
## 30 ( 1 ) "*"          "*"          "*"
## 31 ( 1 ) "*"          "*"          "*"
## 32 ( 1 ) "*"          "*"          "*"
##
##      calculated_host_listings_count dist_to_downtown
## 1 ( 1 ) " "          " "
## 2 ( 1 ) " "          " "
## 3 ( 1 ) " "          " "
## 4 ( 1 ) " "          " "
## 5 ( 1 ) " "          " "
## 6 ( 1 ) " "          " "
## 7 ( 1 ) " "          " "
## 8 ( 1 ) " "          " "
## 9 ( 1 ) " "          " "
## 10 ( 1 ) " "          " "
## 11 ( 1 ) " "          " "
## 12 ( 1 ) " "          " "
## 13 ( 1 ) " "          " "
## 14 ( 1 ) " "          " "
## 15 ( 1 ) " "          " "
## 16 ( 1 ) " "          " "
## 17 ( 1 ) " "          " "
## 18 ( 1 ) " "          " "
## 19 ( 1 ) " "          " "
## 20 ( 1 ) " "          " "
## 21 ( 1 ) " "          " "
## 22 ( 1 ) " "          "*"
## 23 ( 1 ) " "          "*"
## 24 ( 1 ) " "          "*"
## 25 ( 1 ) " "          "*"
## 26 ( 1 ) " "          "*"
## 27 ( 1 ) "*"          "*"
## 28 ( 1 ) "*"          "*"
## 29 ( 1 ) "*"          "*"
## 30 ( 1 ) "*"          "*"
## 31 ( 1 ) "*"          "*"
## 32 ( 1 ) "*"          "*"

```

```
plot(reg.summary$rss , xlab = "Number of Variables", ylab = "RSS", type = "l")
```



```
plot(reg.summary$adjr2 , xlab = "Number of Variables", ylab = " Adjusted RSq", type = "l")
```



```
which.max(reg.summary$adjr2) #adj r2 maximized with 17 predictors
```

```
## [1] 17
```

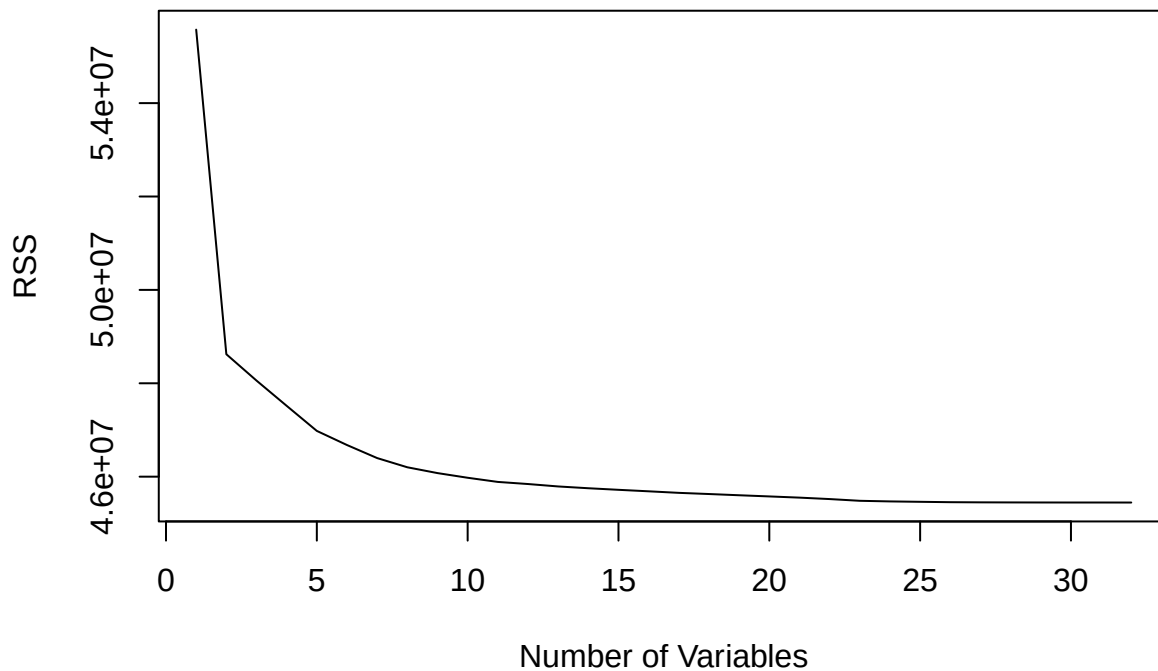
```
coef(regfit.forward, 17)
```

```
##              (Intercept)      neighbourhood_cleansedCarver
##              -117.40655253      62.38548083
##      neighbourhood_cleansedChisago      neighbourhood_cleansedDakota
##              41.52857371      -19.73889481
```

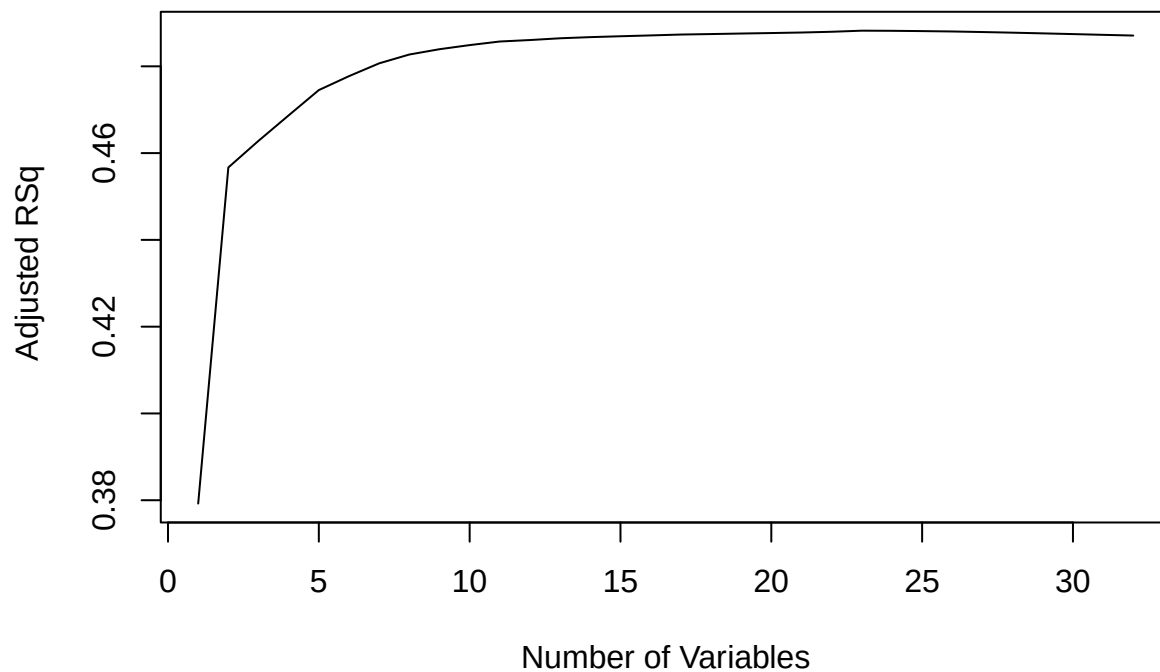
```
##      neighbourhood_cleansedPierce neighbourhood_cleansedSt. Croix
##              43.41153418              107.27141071
## neighbourhood_cleansedWashington      room_typePrivate room
##              73.24376666              -33.37339469
##              room_typeShared room              bathrooms
##              122.71566563              98.36530029
##              bedrooms              accommodates
##              -7.50850254              14.50028370
##              availability_365              number_of_reviews
##              -0.05740388              -0.10284857
##              review_scores_cleanliness              host_is_superhost
##              26.50081718              -16.73441359
##              instant_bookable              host_identity_verified
##              -10.94555389              -21.39828341
```

#Deleted 4 influential points

```
regfit.forwardm4 = regsubsets(price ~ ., data = train.delete, nvmax = 32, method = "forward")
reg.summarym4 = summary(regfit.forwardm4)
plot(reg.summarym4$rss , xlab = "Number of Variables", ylab = "RSS", type = "l")
```



```
plot(reg.summarym4$adjr2 , xlab = "Number of Variables", ylab = " Adjusted RSq", type = "l")
```



```
which.max(reg.summarym4$adjr2) #adj r2 maximized with 23 predictors
```

```
## [1] 23
```

```
coef(regfit.forwardm4, 23)
```

```
##              (Intercept)      neighbourhood_cleansedCarver
##              -177.84245552              79.95877125
##      neighbourhood_cleansedChisago      neighbourhood_cleansedHennepin
##              31.17226208              22.61957926
##      neighbourhood_cleansedLe Sueur      neighbourhood_cleansedMille Lacs
##              24.77499483              -51.72505574
##      neighbourhood_cleansedPierce      neighbourhood_cleansedRamsey
##              42.66700897              16.88527977
##      neighbourhood_cleansedSherburne      neighbourhood_cleansedSt. Croix
##              -31.89580102              120.32276506
##      neighbourhood_cleansedWashington      room_typePrivate room
##              78.13752998              -34.15494728
##              room_typeShared room      bathrooms
##              123.00404359              80.87141366
##              bedrooms      accommodates
##              -3.51355337              15.43774229
##              availability_365      number_of_reviews
##              -0.02503987              -0.11684131
##      review_scores_cleanliness      review_scores_location
##              20.52174919              10.90894767
##              host_is_superhost      instant_bookable
##              -11.19859449              -6.54513966
##      host_identity_verified      dist_to_downtown
##              -8.92527984              1.00412364
```

```
#Linear model (subset)
```

```

#Full dataset
part_lm = lm(price ~ neighbourhood_cleansed + room_type + bathrooms + bedrooms + number_of_reviews + ac
part_lm.ypred = predict(part_lm, train)
part_lm.mse = mean((part_lm.ypred - train$price)^2)
paste0("Partial linear model RMSE: ", round(sqrt(part_lm.mse),2))

## [1] "Partial linear model RMSE: 168.29"

#Deleted 4 points
part_lmm4 = lm(price ~ neighbourhood_cleansed + room_type + bathrooms + bedrooms + number_of_reviews + a
part_lm.ypredm4 = predict(part_lmm4, train.delete)
part_lm.msem4 = mean((part_lm.ypredm4 - train.delete$price)^2)
paste0("Partial linear model RMSE (-4): ", round(sqrt(part_lm.msem4),2))

## [1] "Partial linear model RMSE (-4): 122.59"

#GLM with log link (subset)
log_part_glm = glm(price ~ neighbourhood_cleansed + room_type + bathrooms + bedrooms + number_of_reviews
log_part_glm.ypred <- predict(log_part_glm, train, type = "response")
log_part_glm.mse = mean((log_part_glm.ypred - train$price)^2)
paste0("Partial GLM with log link RMSE: ", round(sqrt(log_part_glm.mse),2))

## [1] "Partial GLM with log link RMSE: 163.43"

#Deleted 4 influential points
log_part_glmm4 = glm(price ~ neighbourhood_cleansed + room_type + bathrooms + bedrooms + number_of_revi
log_part_glm.ypredm4 <- predict(log_part_glm, train.delete, type = "response")
log_part_glm.msem4 = mean((log_part_glm.ypredm4 - train.delete$price)^2)
paste0("Partial GLM with log link RMSE (-4): ", round(sqrt(log_part_glm.msem4),2))

## [1] "Partial GLM with log link RMSE (-4): 118.36"

#Ridge and lasso regression (subset) w/CV
set.seed(1)
#Full dataset
x2 = model.matrix(price ~ neighbourhood_cleansed + room_type + bathrooms + bedrooms + number_of_reviews

part_ridge.cv = cv.glmnet(x2,y,alpha=0)
part_best.ridge.lambda = part_ridge.cv$lambda.min
part_ridge_model = glmnet(x2, y, alpha=0, lambda = part_best.ridge.lambda)
part_ridge.ypred = predict(part_ridge_model, newx = x2, lambda = part_best.ridge.lambda)
part_ridge.mse = mean((part_ridge.ypred - y)^2)
paste0("Partial ridge RSME: ", round(sqrt(part_ridge.mse), 2))

## [1] "Partial ridge RSME: 168.44"

part_lasso.cv = cv.glmnet(x2,y,alpha=1)
part_best.lasso.lambda = part_lasso.cv$lambda.min
part_lasso_model = glmnet(x2, y, alpha=1, lambda = part_best.lasso.lambda)
part_lasso.ypred = predict(part_lasso_model, newx = x2, lambda = part_best.lasso.lambda)
part_lasso.mse = mean((part_lasso.ypred - y)^2)
paste0("Partial lasso RSME: ", round(sqrt(part_lasso.mse), 2))

## [1] "Partial lasso RSME: 168.43"

#4 influential points removed
x2m4 = model.matrix(price ~ neighbourhood_cleansed + room_type + bathrooms + bedrooms + number_of_reviews

```



```

part_ridge.cvm4 = cv.glmnet(x2m4,ym4,alpha=0)
part_best.ridge.lambdam4 = part_ridge.cvm4$lambda.min
part_ridge_modelm4 = glmnet(x2m4, ym4, alpha=0, lambda = part_best.ridge.lambdam4)
part_ridge.ypredm4 = predict(part_ridge_modelm4, newx = x2m4, lambda = part_best.ridge.lambdam4)
part_ridge.msem4 = mean((part_ridge.ypredm4 - ym4)^2)
paste0("Partial ridge RSME (-4): ", round(sqrt(part_ridge.msem4), 2))

## [1] "Partial ridge RSME (-4): 122.8"

part_lasso.cvm4 = cv.glmnet(x2m4,ym4,alpha=1)
part_best.lasso.lambdam4 = part_lasso.cvm4$lambda.min
part_lasso_modelm4 = glmnet(x2m4, ym4, alpha=1, lambda = part_best.lasso.lambdam4)
part_lasso.ypredm4 = predict(part_lasso_modelm4, newx = x2m4, lambda = part_best.lasso.lambdam4)
part_lasso.msem4 = mean((part_lasso.ypredm4 - ym4)^2)
paste0("Partial lasso RSME (-4): ", round(sqrt(part_lasso.msem4), 2))

## [1] "Partial lasso RSME (-4): 122.86"

```

```
##Predictions of best model (RF)
```

```
test_preds = predict(rf_model, test)
predictions = data.frame(id = test$ID, prediction = test_preds)
predictions = round(predictions,2)
write.csv(predictions, "testing_predictions_KWEE_ELIZABETH_ELK204.csv", row.names=FALSE)
head(predictions)
```

```
##   id prediction
## 1  1      65.31
## 2  2     109.08
## 3  3     201.78
## 4  4     125.29
## 5  5      81.46
## 6  6     408.17
```